

Background

Splice sites are short DNA sequences found in eukaryotic genes that are crucial for proper messenger RNA (mRNA) splicing. These sites are located at the boundaries between exons and introns and consist of two essential components: the 5' splice site (acceptor site) and the 3' splice site (donor site)¹.

The 5' splice site is typically characterized by a highly conserved GT dinucleotide sequence located at the exon-intron junction. Additionally, this site contains an adjacent polypyrimidine tract, which serves as a binding site for the U2 small nuclear ribonucleoprotein (snRNP) complex during spliceosome assembly².

The 3' splice site, on the other hand, is marked by a conserved AG dinucleotide sequence located at the intron-exon junction. This site also contains a downstream branch point sequence that plays a role in defining the site of the intron-exon boundary during splicing³.

Together, these splice sites, along with various other splicing regulatory elements, form a complex network of interactions that orchestrate the precise removal of introns and the ligation of exons to form mature mRNA transcripts¹. Understanding the function and regulation of splice sites is essential for deciphering the complex mechanisms of gene expression and the pathogenesis of numerous genetic diseases.

Mutations in the donor or acceptor splice sites can disrupt the splicing process, leading to the exon skipping, intron retention, or the activation of cryptic splice sites, which can alter the reading frame of the mRNA and result in truncated or non-functional proteins. These mutations can lead to a variety of genetic disorders, including cystic fibrosis⁴, beta-thalassemia⁵, and spinal muscular atrophy⁶. Alternative splicing can lead to isoforms of a protein having different functional capabilities, such as B cell activation via *TAC1*⁷.

Therefore, understanding the mechanism of splicing and the recognition of splice sites is crucial for understanding gene expression and for the development of therapeutic strategies for genetic diseases caused by splicing defects

Rationale

Splice sites are critical regulatory elements of gene expression that have a profound impact on the production of functional proteins. Identifying and predicting splice sites is of paramount importance for understanding the molecular mechanisms underlying various diseases, including genetic disorders. Accurate prediction of splice sites is necessary for the diagnosis and treatment of these diseases, as well as the development of effective therapies. Failure to identify mutations in splice sites can result in incorrect genetic diagnoses, and misinterpretation of the disease mechanism, leading to ineffective treatments. Therefore, the development of accurate predictive models for splice sites is crucial for advancing our understanding of genetic diseases and improving patient outcomes.

Alternative splicing is a critical process in gene expression regulation, and its dysregulation can lead to various diseases. For instance, spinal muscular atrophy (SMA) is a genetic disease caused by the loss of survival motor neuron 1 (SMN1) gene function, resulting in degeneration of motor neurons in the spinal cord. However, a related gene, SMN2, can partially compensate for SMN1 loss by producing a truncated, unstable protein. The SMN2 gene undergoes alternative splicing, leading to variable inclusion of exon 7. Exon 7 skipping results in an unstable protein that fails to rescue the SMN1 loss, leading to SMA. On the other hand, higher inclusion of exon 7 in SMN2 can produce a more stable protein and less severe disease phenotype⁶.

Another example is cystic fibrosis (CF), a common inherited disease caused by mutations in the cystic fibrosis transmembrane conductance regulator (CFTR) gene. The 3849+10kb C>T mutation leads to the creation of a new acceptor splice site in intron 19, resulting in the inclusion of a pseudoexon and premature termination of the CFTR protein. This aberrant splicing event causes CFTR misfolding, resulting in its degradation and leading to the disease phenotype⁴.

Therefore, precise identification and prediction of splicing events, including alternative splicing, are critical to understanding and diagnosing genetic diseases.

Alternative splicing can also impact functional phenotype in the absence of mutation. The human *TACI* gene undergoes alternative splicing to produce short and long isoforms (TACI-S and TACI-L). Expression of different isoforms in humans provides unique effects on B cell maturation, activation, ligand binding capability and subcellular localization⁷. Enhanced accuracy in the prediction of splice sites can also drive the discovery of novel biology, identifying the impact of isoforms, due to alternative splicing.

Accurate prediction of splice sites is crucial for understanding the genetic basis of diseases and developing effective treatments. Improving the accuracy of splice site prediction can enhance our ability to identify disease-causing mutations, predict the effects of genetic variations, discover novel biology and develop targeted therapies. We aim to develop a machine learning model that can accurately predict novel splice sites from DNA sequences.

Approach

Dataset: We used the Homo Sapiens Splice Sites Dataset (HS³D) to build our models which is comprised of human genomic DNA introns and exons from the Primate Sequences Division of GenBank Rel. 123. This dataset contains 2,796 true positive donor splice site regions, 2,880 true positive acceptor splice site regions, 271,937 false donor splice site regions, and 329,374 false acceptor splice site regions. Each entry in the dataset is a 140 nucleotide (nt) genomic sequence, which consists of the four genomic bases (A, C, G and T) and H, which we treated as an unknown base. For the donor splice site regions, each sequence, both true and false regions, have a GT sequence at positions 70 and 71. For the acceptor splice site regions, each sequence, both true and false regions, have an AG sequence as positions 69 and 70. The

distribution of each class is shown in **Figure 1**. Both datasets are very imbalanced with about 100 false sites for every 1 true site.

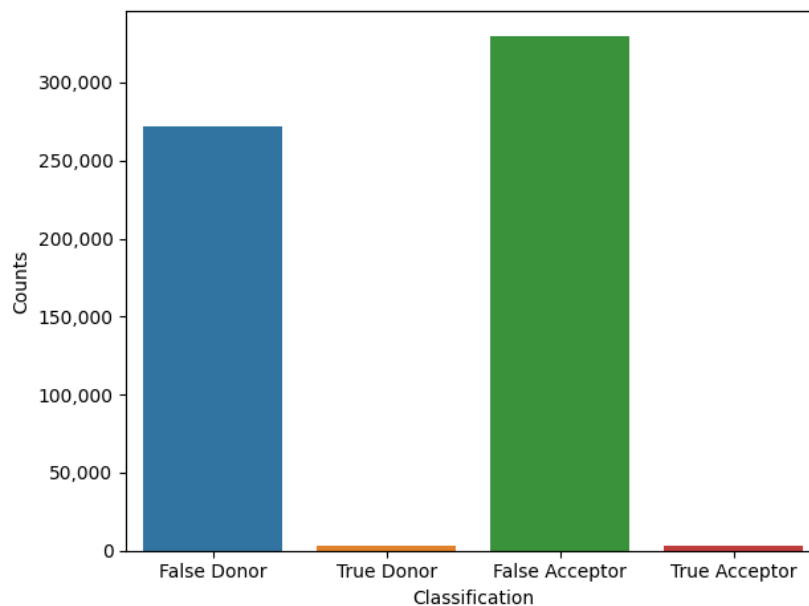


Figure 1. Distribution of classes in the HS³D dataset

Data splitting: The two datasets were split into training and testing sets using stratification on the target column, so the resulting splits would have the same proportion of true and false splice sites as the original dataset. 25% of the donor and acceptor site datasets were retained as test sets used in all models below. The remaining 75% of the datasets were used for training and were further split into training and validation sets during training. This split was done randomly for each model so different training/validation splits were used throughout model testing.

Training two separate models: We decided to train two separate models, one to classify donor sites and the other to classify acceptor sites. We chose this approach as motifs which are important for donor sites might differ from those that are important for acceptor sites. Additionally, there is an established difference between canonical donor and acceptor sites sequences: the presence of GT or AG. As a result, two separate models can potentially learn differences between donor and acceptor sites better than a single general model that attempts to learn patterns for both sites.

One-hot encoding: Each entry in the dataset is a 140 nt sequence composed of 5 letters: the bases A, C, G and T and H which we presume to mean unknown base. To apply the following machine learning classifiers, this needs to be converted into a numerical format appropriate for a non-ordinal categorical variable at each base. In most cases, we used one-hot encoding for this. This converts each base into a numerical vector. The following one-hot encoding was used:

A = [1,0,0,0]

C = [0,1,0,0]

G = [0,0,1,0]

$T = [0,0,0,1]$

$H(\text{unknown}) = [0,0,0,0]$

The 140 nucleotide sequence thus becomes a $[4 \times 140]$ array. One-hot encoding was used unless otherwise noted below.

Random Forest Classifier: A random forest classifier (RFC) is a commonly used classification tool used in machine learning. The model within the classifier consists of an ensemble of decision trees that combine making a “random forest”. The classifier uses bagging which randomly samples from the dataset with replacement to train each decision tree. It also utilizes feature randomness where at each node, a random subset of features is chosen. This ensures that the decision trees are uncorrelated. The RFC was trained on the full training set, separately for donor and acceptor sites. We flattened the one-hot encoding so that each entry had 560 features corresponding to a position-nucleotide pair (ex: 1_A, 1_C, 2_T, etc). Each feature is binary, 1 representing that it has that base at that position in the sequence. The classifier was applied to the test set to evaluate performance. We built and trained these models using the scikit-learn package of Python. To handle class imbalance specific to scikit-learn, we tested undersampling the majority class to match the minority class. The test set was not sampled.

Handling Class Imbalance in PyTorch: Besides the RFC, all other models were built using PyTorch. As shown in **Figure 1**, there is extreme class imbalance in this dataset, both for donor and acceptor sites. For all PyTorch models, to account for this, we used WeightedRandomSampler to sample the data according to weights inversely proportional to the class counts. This oversamples the minority classes and leads to more balanced batches while training. The validation and test sets were not sampled. Additionally, in order to best evaluate the performance of the model, we used the macro-average F1 score (F1-macro). This more heavily considers the performance of the model on the positive class over just considering accuracy. The model can achieve 99% accuracy by predicting everything as a false splice site. The F1-macro score better reflects the performance on the positive class. We used the F1-macro score during training to select the best performing model seen over the epochs.

Additional model training parameters in PyTorch: For each model implemented in PyTorch, we used certain model parameters and features consistently unless otherwise stated below. We used the SGD optimizer with momentum set to 0.9 and weight decay set to 0.0001. Momentum sets the weight of the exponential moving average during gradient descent to more efficiently minimize loss and be less subject to noisy trajectories, allowing the model to converge faster. Weight decay is a regularization strategy that adds a penalty to the loss function, usually the L2 norm of the model weights. This maximizes the generalizability of the model on an unknown testing set. The initial learning rate varied. We utilized a learning rate scheduler, specifically ReduceLROnPlateau, which decreases the learning rate after a number of epochs set by the patience parameter in which validation loss did not decrease. We used cross entropy loss as the loss function. Additionally, we implemented an early stopping criteria so the model stops training after a number of epochs set by the patience parameter in which validation loss did not decrease to prevent overfitting.

Convolutional Neural Networks: A convolutional neural network (CNN) is a widely used model especially optimized for feature extraction. CNNs are largely effective when trained on large datasets, which genomic data is. While CNNs were originally used for computer vision, and trained on 2D images with three color channels (R,G,B), they can be applied to 1D data, such as genomic data, which is a 1D sequence made up of four channels, which are the four nucleotides (A,C,T,G). Here, we conceptualized our CNN to address a 1D binary classification problem with four channels, to classify positive vs negative splice sites.

Here, we use convolutional layers to extract features from DNA sequences. Extracted features can be thought of as motif discovery, to identify novel motifs mediating splice donor and acceptor activity. Here, we tested window sizes of 1, 5, 7, and 15, eventually choosing a final model using 15 in order to better identify complex motif patterns within a large base pair window. We also incorporated max-pooling which outputs only the maximum value from each convolutional layer, which theoretically encodes whether or not the identified motif is within the target sequence. We incorporate dropout to minimize overfitting. Various data have shown that deeper CNNs are not necessarily more effective in the case of DNA motif analysis⁹. Additionally, outputs from deeper CNNs are harder to conceptualize biologically, as the features learned become higher order and difficult to visualize. Therefore, we trained a two layer CNN. The architecture of this model is shown in **Table 1**. We visualized the output from the first layer, as it maintained the shape of four nucleotides and a 15 base pair motif, across multiple filters. We have visualized the weights for each nucleotide across each filter in the first convolutional layer in our results. This gives us an idea of the motifs the CNN identifies as contributing to donor or acceptor splice site categorization. Motifs more associated with the positive class will be weighted highly by the first convolutional layer. Therefore, we can pull out the nucleotide sequences weighted highly by the model and search for common motifs that may represent novel splice site predictor motifs.

Table 1. CNN architecture summary.

Layer	Parameters
Conv1d	Number of nucleotides (4), number of filters (32), kernel size (15)
ReLU	
MaxPool1d	Max pool of window size 3
Dropout	p=0.2
Conv1d	Number of filters (32), Kernel size (32)
ReLU	
MaxPool1d	Max pool of window size 3
Dropout	p=0.2
Linear	

LSTM: We hypothesized that a Long Short Term Memory (LSTM) Recurrent Neural Network (RNN) might perform well in these models. RNNs are designed for sequential or time series data and use information from prior inputs to influence a current input and output. RNNs have hidden states that store information of the previous entries in the sequence or time series. LSTMs are a type of RNN architecture, which consists of “cell states.” They were designed to retain “long-term memory” to handle longer sequences and also to combat the vanishing and exploding gradient problems found in traditional RNNs.

In this model, instead of one-hot encoding, we used embeddings to convert the sequence into vectors representing the bases. First, we converted each base into a numerical category from the letter. Each entry in the dataset became a vector of size 140 with a number 0 through 4 representing the base at each position. This was used as input into the model. The model first contains an embedding layer which is trained during model training to represent each base as a vector.

The model architecture summary is shown below in **Table 2**. First, there is the embedding layer. This takes in two key parameters. The vocab size refers to how many different “words” are seen. In this case each base is a word. This is set to 5 to account for “H” which is the unknown base. The other parameter is the embedding size which is the size of the output embedding vector for each “word”. The output of this layer is input into the LSTM layer. The input size of this layer is the embedding size of the embedding layer. The hidden size is the number of features in the hidden state for the LSTM. The last hidden state is used as input into a ReLU activation layer and then a fully connected (FC) layer with a given output size, FC size. This output is used in another ReLU layer and then the second and final fully connected layer. The input size of this is the output size of FC1 and the output size is the number of classes, which is 2 (true splice site or false splice site). We tested a wide range of different values for the hyperparameters listed below and picked the best hyperparameters based on the F1-macro scores.

Table 2. LSTM architecture summary.

Layer	Parameters
Embedding	Vocab Size = 5, Embedding Size
LSTM	Input Size = Embedding Size, Hidden Size
ReLU	—
Fully Connected (FC1)	Input Size = Hidden Size, Output Size = FC Size
ReLU	—
Fully Connected (FC2)	Input Size = FC Size, Output Size = Number of Classes

Additionally, we hypothesized that a bidirectional LSTM could perform better than a standard LSTM. In this case, the input goes in both directions in the LSTM, using information from both sides of the sequence. This could be useful for classifying a DNA sequence where the sequence after the splice site might be just as influential or more than the sequence before it. The model architecture summary is shown below in **Table 3**. It is the same as the unidirectional LSTM above, except that the LSTM layer is bidirectional. Additionally, the hidden states of each direction are concatenated for FC1. The same set of hyperparameter combinations were tested on this model.

Table 3. Bidirectional LSTM architecture summary.

Layer	Parameters
Embedding	Dictionary Size = 5, Embedding Size
LSTM	Input Size = Embedding Size, Hidden Size, Bidirectional = True
ReLU	—
Fully Connected (FC1)	Input Size = Hidden Size*2, Output Size = FC Size
ReLU	—
Fully Connected (FC2)	Input Size = FC Size, Output Size = Number of Classes

CNN/LSTM

We then built models that combine CNN, LSTM and FC layers. The CNN layers can be used to extract features from one-hot encoded sequences, the LSTM can capture the sequences' order, and then the FC layers are used for classification. The architecture of the first model we used and the parameters of the model are shown in **Table 4**. Here, we use the one-hot encoded sequencing rather than the embeddings used in the LSTM models above. The model has a first convolutional layer with 4 input channels, one for each of the bases in the one-hot encoding. Then there is a dropout layer before a second convolutional layer. In the second layer, the input channel is set to the number of filters. For both convolutional layers, padding was used to keep the output of the convolutional layers the same length as the original input. The padding is set to $padding = floor(kernel\ size/2)$, with the kernel size being an odd integer value. There is another dropout layer. The output of that is used as the input to a bidirectional LSTM. Here, the input size for the LSTM is the number of filters. The LSTM layer is followed by ReLU and FC layers for classification. We tested a wide range of different values for the hyperparameters listed below and picked the best hyperparameters based on the F1-macro scores.

Table 4. CNN/Bidirectional LSTM architecture summary.

Layer	Parameters
Convolution 1D (Conv1)	Input Channels = 4, Number of Filters, Kernel Size, Padding
Dropout	Probability
Convolution 1D (Conv2)	Input Channels = Number of Filters, Number of Filters, Kernel Size, Padding
Dropout	Probability
LSTM	Input Size = Number of Filters, Hidden Size, Bidirectional = True
ReLU	—
Fully Connected (FC1)	Input Size = Hidden Size * 2, Output Size = FC Size
ReLU	—
Fully Connected (FC2)	Input Size = FC Size, Output Size = Number of Classes

We also tested an alternative CNN/Bidirectional LSTM architecture shown in **Table 5**, which is the same as the one above but with max pooling layers added after each of the two convolutional layers. The max pool layers pool the maximum value for each patch of a feature map using a pooling kernel size and stride hyperparameters. This retains the most prominent features in each patch from the feature map. Again, we tested a range of hyperparameters for this model.

Table 5. CNN/Bidirectional LSTM with max pooling architecture summary.

Layer	Parameters
Convolution 1D (Conv1)	Input Channels = 4, Number of Filters, Kernel Size, Padding
Max Pooling 1D	Pooling Kernel Size, Stride
Dropout	Probability
Convolution 1D (Conv2)	Input Channels = Number of Filters, Number of Filters, Kernel Size, Padding
Max Pooling 1D	Pooling Kernel Size, Stride
Dropout	Probability

LSTM	Input Size = Number of Filters, Hidden Size, Bidirectional = True
ReLU	—
Fully Connected (FC1)	Input Size = Hidden Size * 2, Output Size = FC Size
ReLU	—
Fully Connected (FC2)	Input Size = FC Size, Output Size = Number of Classes

Results:

Random Forest Classifier (RFC): First, we tested an RFC on the original training set without accounting for class imbalance. For both donor and acceptors, the classifiers have high accuracies: 99% and 99.1% respectively. However, the F1-macro score is 0.55 and 0.54 respectively. Looking at the confusion matrices in **Table 6**, the classifier predicts most of the true splice sites as false splice sites. The recall for both models on the positive class is 0.05. With the class imbalance, the accuracy is still very high because it is able to correctly predict about all of the false sites.

Table 6. Confusion matrices RFC.

Donors	Predicted False	Predicted True	Acceptors	Predicted False	Predicted True
False	67,983	2	False	82,344	0
True	662	37	True	687	33

In order to improve performance on the positive class, we used undersampling. For undersampling, we randomly sampled the false splice sites in each dataset to match the number of true splice site examples only in the training set but left the test set the same. While the accuracy of each model decreases (91.3% and 86.6% respectively), the recall score of each model on the positive class increases to 0.93 and 0.91 respectively. In the confusion matrices (**Table 7**), most true splice sites were predicted to be true. However, the F1-macro scores both increase slightly (to 0.57) because the increase in recall comes at the expense of precision. A lot of false splice sites are predicted to be true splice sites in both models. **Figure 3** shows the precision-recall curve for each of the models, which has a low area under the curve. The area under the ROC curves (**Figure 2**) is high for both models at 0.97 and 0.96 respectively. By undersampling, we lose a lot of the data for the false splice sites, which could potentially impact performance.

Table 7. Confusion matrices undersampled RFC.

Donors	Predicted False	Predicted True	Acceptors	Predicted False	Predicted True
False	62,088	5,897	False	71,293	11,051
True	46	653	True	68	652

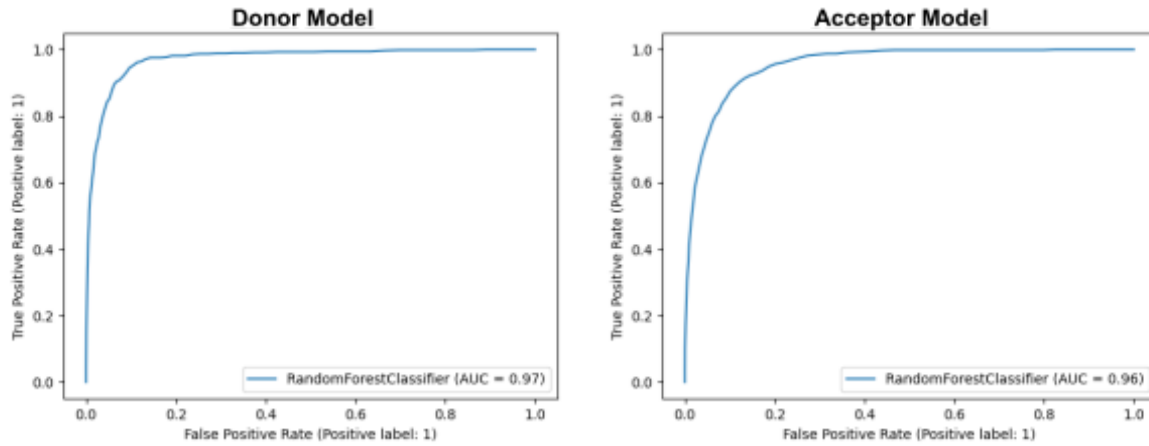


Figure 2. ROC curve for RFCs for donor sites (left) and acceptor sites (right) with AUC (0.97 and 0.96 respectively)

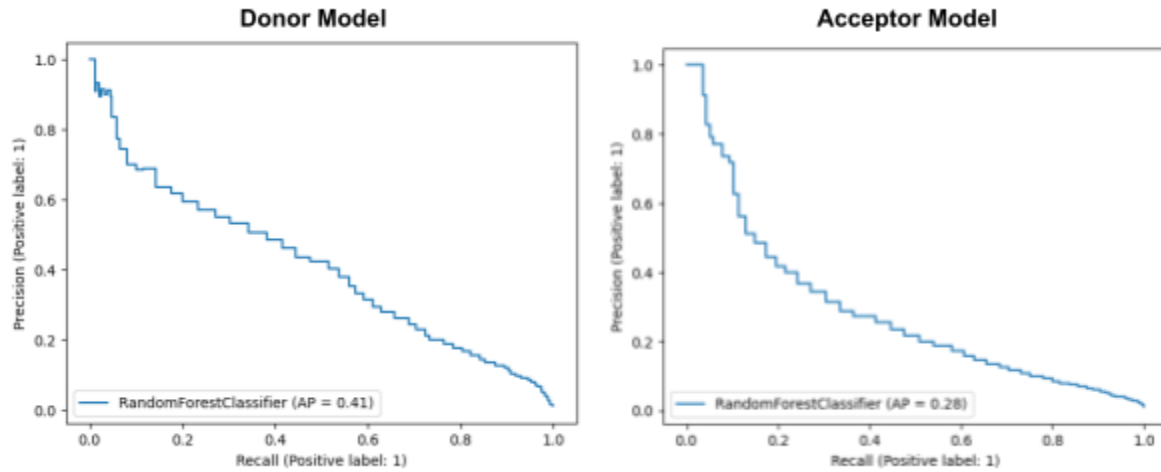


Figure 3. Precision-recall curves for RFCs for donor sites (left) and acceptor sites (right) with the area under the curve (AP, 0.41 and 0.28 respectively)

For each of these models, we extracted the feature importances. Due to the one-hot encoding, each base at each position is a separate feature. **Figure 4** shows the feature importances of each base stacked for each position for both models. In both models, the highest feature importances surrounds the GT or AG sequence where the splicing occurs. In the donor model,

there are slightly higher feature importances to the right of the GT in the sequence. In the acceptor model, bases to the left of the AG in the sequence have higher importances. This might indicate which regions of the sequence surrounding these sites are most important for determining whether or not it is a splice site. We notice that donor and acceptor sites seem to have different feature importance plots, which validates the idea to train separate models to be able to differentiate patterns and motifs specific to each type of splice site.

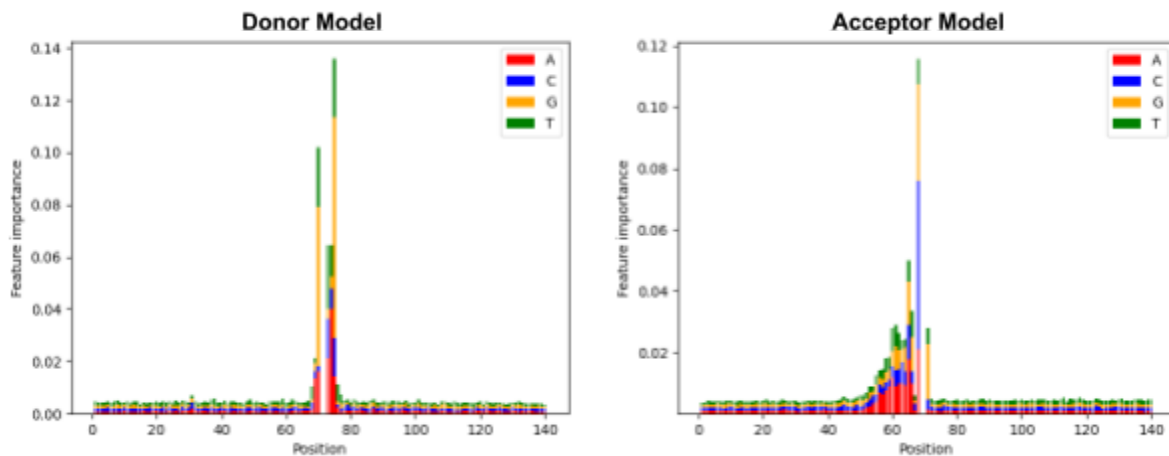


Figure 4. Feature importances at each position stacked for each of the four bases for the donor RFC model (left) and acceptor RFC model (right).

Convolutional Neural Networks

We next tested a convolutional neural network, using the logic that traditional convolutional layers could be applied in 1D to a sequence of nucleotide bases, with 4 channels representing the 4 nucleotides, A, C, T, and G. We tested models as described in our methods section. Here, our models performed with an accuracy lower than that of our random forest classifier, with a best accuracy of 0.875 and 0.914 for the donor and acceptor models, respectively. Our best F1 macro was 0.551. However, the AUCs were slightly better than the random forest classifier, at 0.98 for both models (**Figure 5**). A lot of false splice sites are predicted to be true splice sites in both models. **Figure 6** shows the precision-recall curve for each of the models.

Table 8. Confusion matrices for CNNs.

Donors	Predicted False	Predicted True	Acceptors	Predicted False	Predicted True
False	59,484	8,501	False	75,262	7,082
True	93	606	True	94	626

Donor model (left), Acceptor model (right)

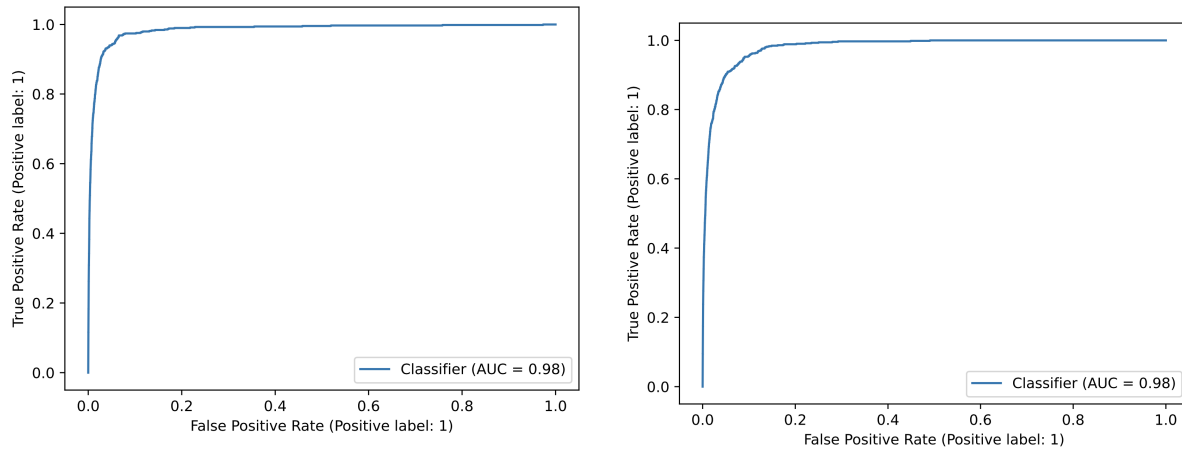


Figure 5. ROC curve for CNN model for donor sites (left) and acceptor sites (right) with AUC (Both 0.98)

Donor model (left), Acceptor model (right)

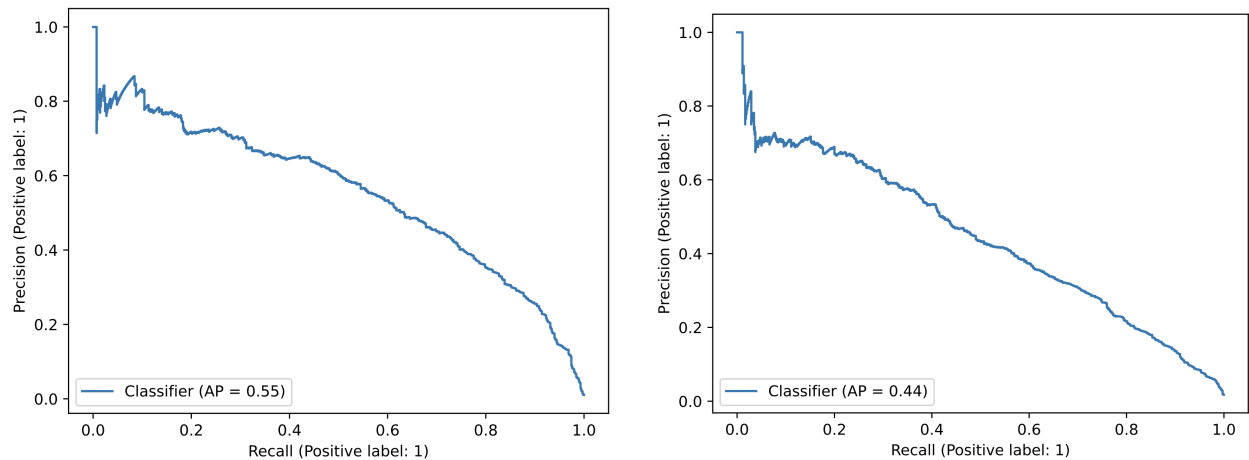


Figure 6. Precision-recall curves for CNN models for donor sites (left) and acceptor sites (right) with the area under the curve (AP, 0.55 and 0.44 respectively)

However, the advantage of a CNN is it allows for visual representations of the learned parameters as determined by the weights given to the first convolutional layer. As the model takes in an input with 4 channels, the weights given to each channel can be associated with the nucleotide's relative importance to the classification task.

To visualize this, the weights of the first layer of the CNN were used to visualize the motifs prioritized by the neural network as positively or negatively associated with donor or acceptor splice sites. The visualization of these motifs is as shown below in **Figures 7** and **8**. These motifs can be interpreted as DNA sequences that the convolutional layer extracts as features

that are particularly associated with donor or acceptor splice sites. (The prioritized sequences of these visualizations are presented in **Supplementary Table 1**).

Donors

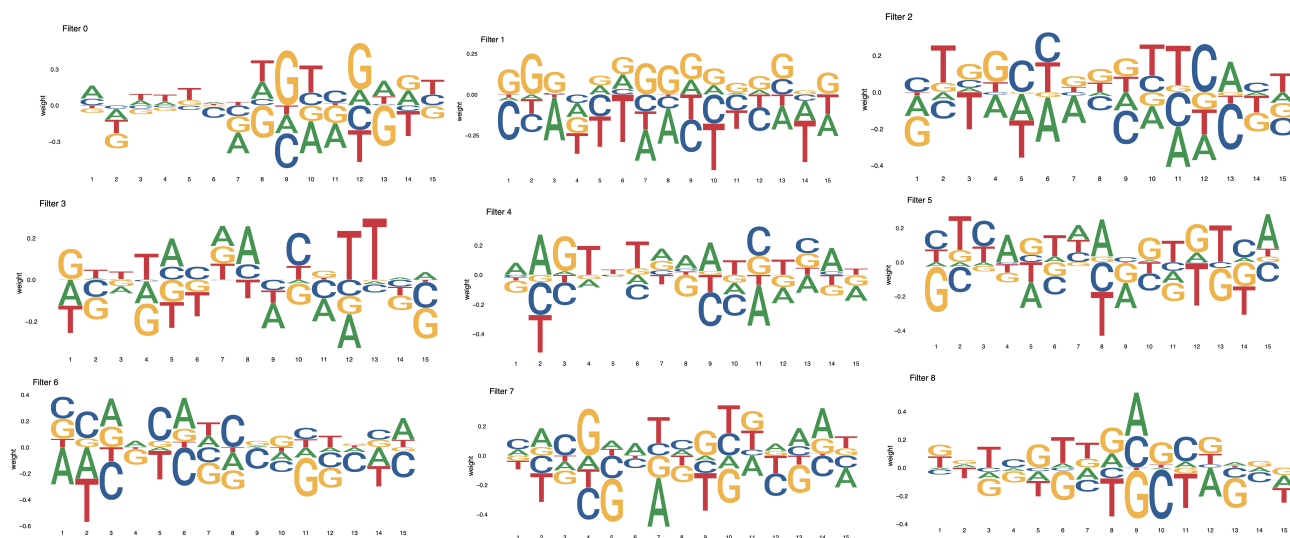


Figure 7. Visualization of various CNN filters for donor site model.

Acceptors

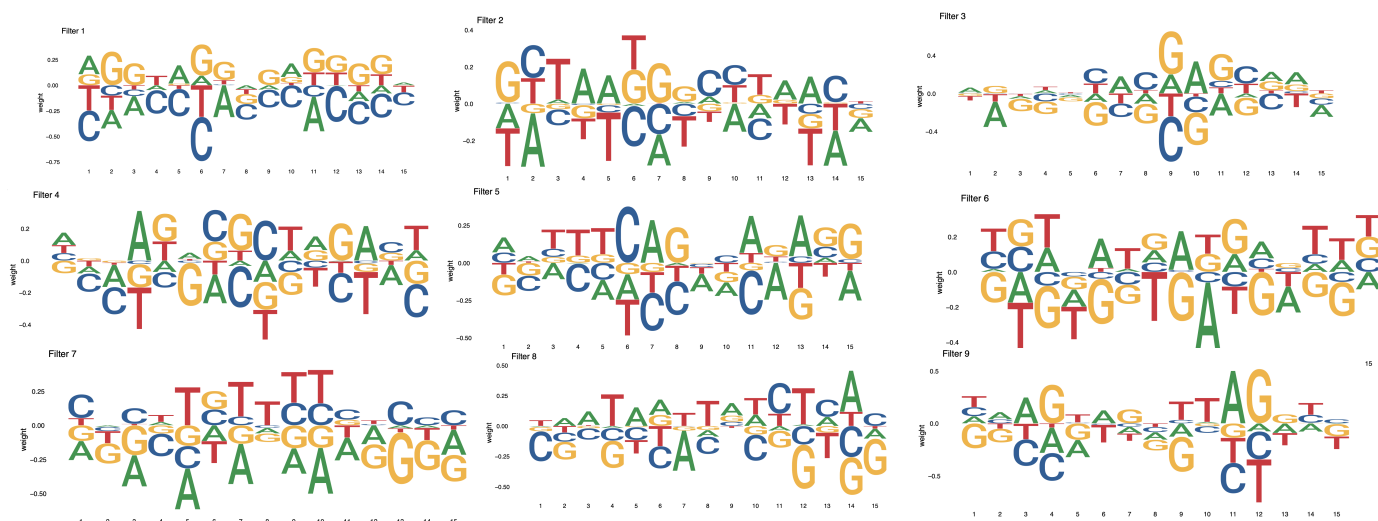


Figure 8. Visualization of various CNN filters for acceptor site model.

As each filter extracts slightly different sites, we can next use motif enrichment analyses to pull out common nucleotide sequences associated with splice sites across many filters. This could identify any common motifs that may represent novel motifs that predict and regulate splice site behavior. To do that, we used the MEME¹⁰ Suite of tools to assess what nucleotide sequence motifs were more enriched in this sample of sequences compared to all eukaryotic sequences. Compared to all common motifs, these motifs were particularly enriched in this sample. Enrichments are shown below in **Figures 9** and **10**.

Donors

Motif 	Logo 	RC Logo 	Score 	Sites 
1-GTWAGT			2.6e-002	5 (16.1%)
2-AKASTMT			9.8e-002	5 (16.1%)
3-CAASCT			9.8e-002	5 (16.1%)
4-CTAGAS			9.8e-002	5 (16.1%)
5-KCMGYAMMG			2.5e-001	2 (6.5%)

Figure 9. Motif enrichment of CNN filters for donor site model.

As none of these donor motifs match known motifs, they can be considered novel motifs that may serve as sequence predictors of donor splice sites.

Acceptors

Motif ?	Logo ?	RC Logo ?	Score ?	Sites ?
1-MTWRYWAK			2.6e-002	7 (22.6%)
2-AGCRTCTGY			5.2e-002	6 (19.4%)
3-AGAGGGGA			9.8e-002	5 (16.1%)
4-CCTA			1.8e-001	9 (29.0%)
5-AWSCG			2.5e-001	2 (6.5%)

Figure 10. Motif enrichment of CNN filters for acceptor site model.

Notably, the second motif discovered in the acceptor sites is similar to the known Ascl2 motif (**Figure 11**). Ascl2 is a developmental regulatory binding site associated with alternative splicing with differential regulation in maternal and paternal alleles⁴¹.

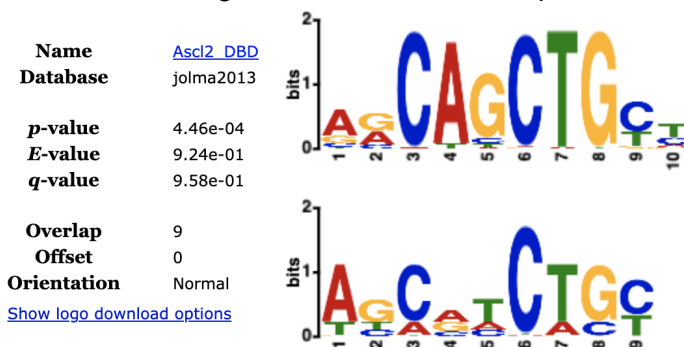


Figure 11. Known Ascl2 motif similar to discovered acceptor site motif.

Additionally, the fifth motif discovered to be enriched contains overlap with many known PAX1, PAX5, PAX6, PAX9 motifs, in addition to the CENPB motif (**Figure 12**).

PAX genes are highly conserved transcription factors that are considered “master regulators” throughout development and tissue function¹². There are 9 human PAX family members. Transcriptional regulation by PAX genes is known to occur via alternative splicing. Therefore, downstream targets of PAX transcription factors and isoforms are of high interest to discover

ramifications of such alternative splicing. Already, PAX proteins have been implicated in various developmental diseases as well as various forms of cancer.

CENPB is another highly conserved DNA binding protein that is involved in centromere formation. CENPB binding sites are highly associated with centromere structures. The CENP family of genes have also been reported to undergo alternative splicing, especially in leukemia¹³, leading to variable protein expression.

Together, these enrichments indicate known alternative splicing pathways are enriched in CNN discovery, suggesting that our CNN model is adept at recognizing motifs relevant to alternative splicing. Additionally, this further indicates that additional motifs identified by the model may represent novel splice sites that may have relevance to future splice site prediction.

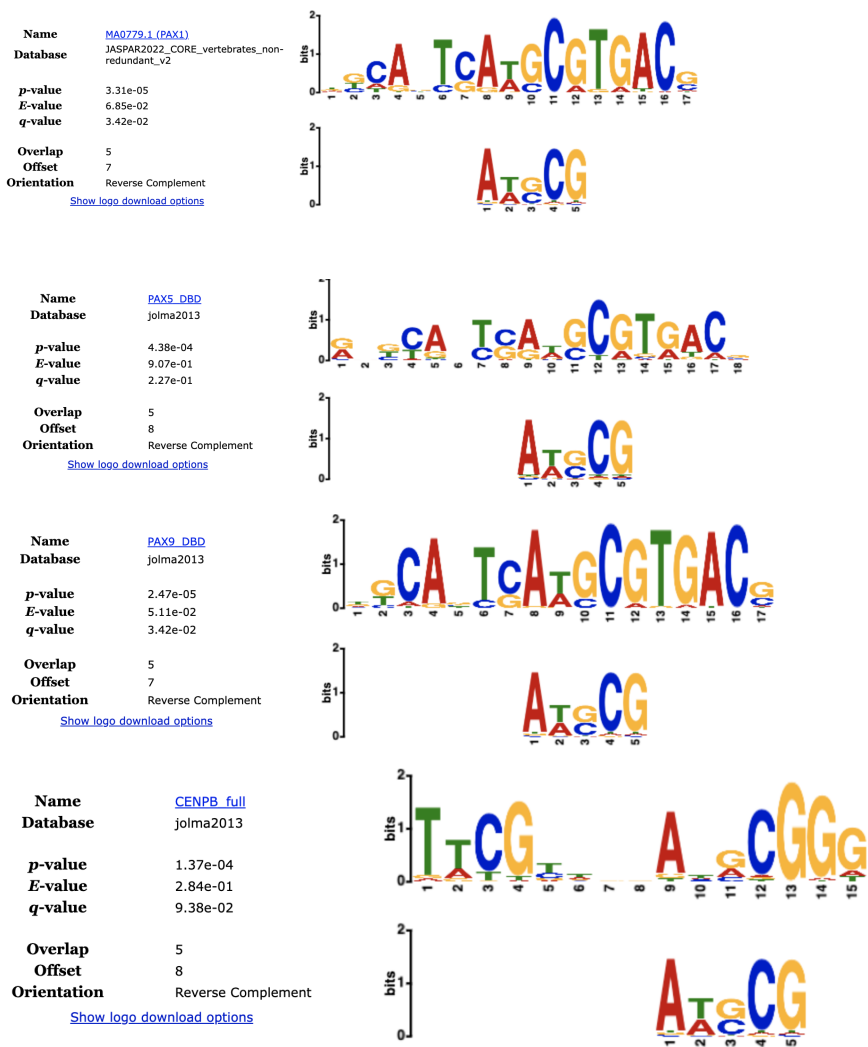


Figure 12. Known motifs that overlap the fifth enriched acceptor site motif.

LSTM: From a hyperparameter search evaluating the best set based on F1-macro scores, we found the best performance for the hyperparameters in **Tables 9** and **11** for unidirectional and

bidirectional LSTM models respectively. The donor and acceptor models had different optimal values for some of the hyperparameters. **Table 9** shows the separate model hyperparameters for donor sites and acceptor sites for the unidirectional LSTM model.

Table 9. Best hyperparameter values for unidirectional LSTM models.

Parameter	Donor Model Parameter Value	Acceptor Model Parameter Value
Initial learning rate	0.1	0.1
Embedding Size	20	20
Hidden Size	64	200
FC Size	128	128

The unidirectional LSTM model for donor sites had a test accuracy of 98.9% and the one for acceptor sites had a test accuracy of 99.1%. **Figure 13** shows the ROC curves for both models with AUC scores of 0.96 for both models. The confusion matrices for these models on the test set are in **Table 10**. Both F1-macro scores are improvements at 0.79 and 0.78 respectively. The precision-recall curves are shown in **Figure 14**.

Table 10. Confusion matrices LSTMs.

Donors	Predicted False	Predicted True
False	67,402	583
True	168	531

Acceptors	Predicted False	Predicted True
False	81,841	503
True	250	470

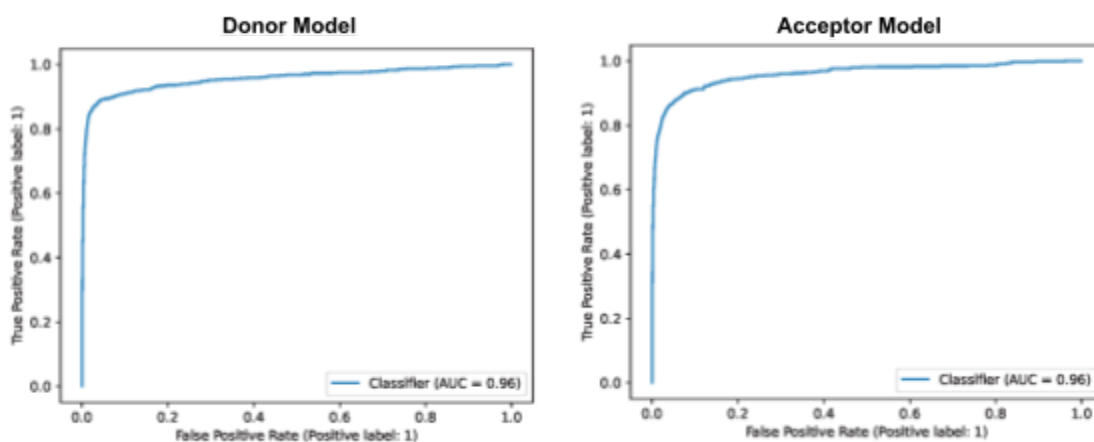


Figure 13. ROC curve for unidirectional LSTM models for donor sites (left) and acceptor sites (right) with AUC (both 0.96).

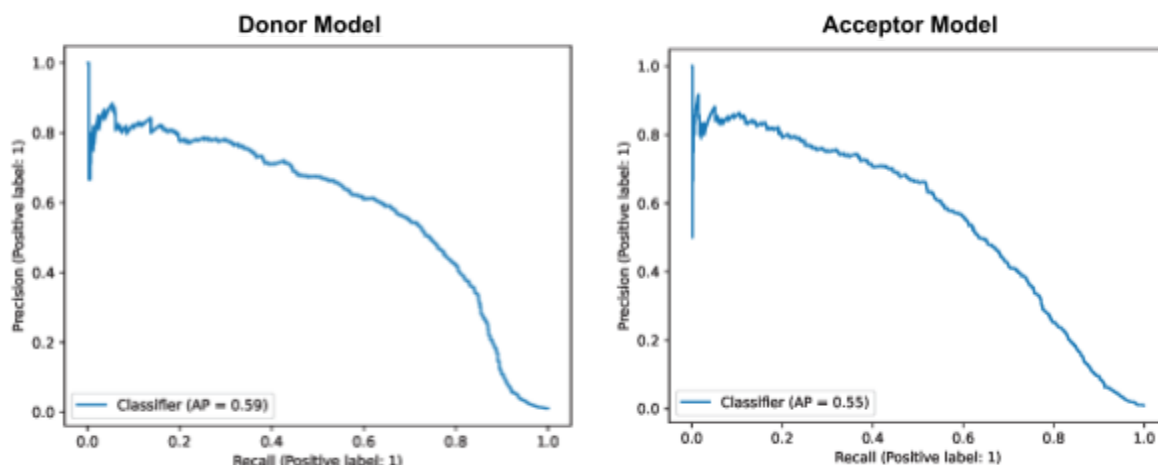


Figure 14. Precision-recall curves for unidirectional LSTM models for donor sites (left) and acceptor sites (right) with the area under the curve (AP, 0.59 and 0.55 respectively).

Table 11 shows the separate model hyperparameters for donor sites and acceptor sites for the bidirectional LSTM model with the best performance.

Table 11. Best hyperparameter values for bidirectional LSTM models.

Parameter	Donor Model Parameter Value	Acceptor Model Parameter Value
Initial learning rate	0.1	0.1
Embedding Size	5	2
Hidden Size	200	200
FC Size	128	128

The bidirectional LSTM models for donor and acceptor sites both had a test accuracy of 99.2% an improvement compared to the unidirectional model. **Figure 15** shows the ROC curves for both models with AUC scores of 0.97 for both models, also a very slight improvement compared to the unidirectional model. The confusion matrices for these models on the test set are in **Table 12**. The F1-macro score for the donor model improved to 0.81 and the score remained the same for the acceptor model at 0.78. The precision-recall curves are shown in **Figure 16**.

Table 12. Confusion matrices bidirectional LSTMs.

Donors	Predicted False	Predicted True
False	67,612	373
True	207	492

Acceptors	Predicted False	Predicted True
False	81,922	422
True	272	448

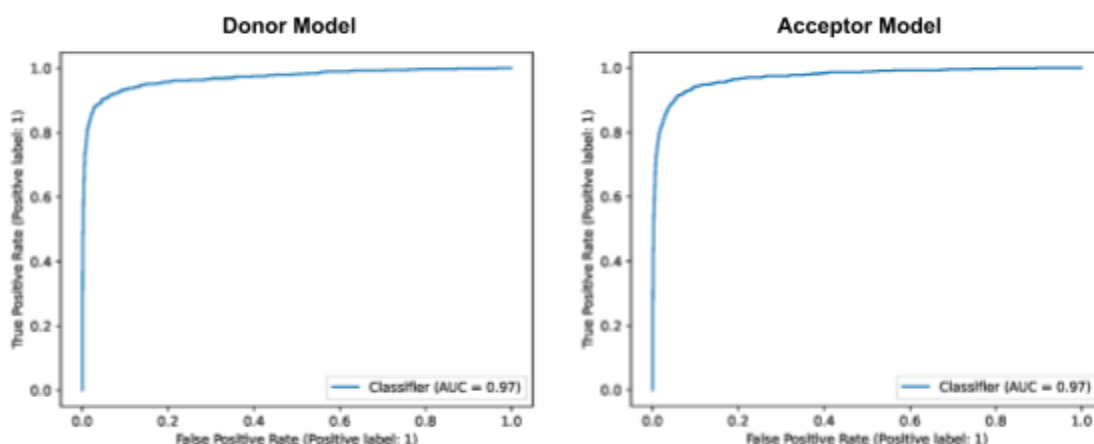


Figure 15. ROC curve for bidirectional LSTM models for donor sites (left) and acceptor sites (right) with AUC (both 0.97).

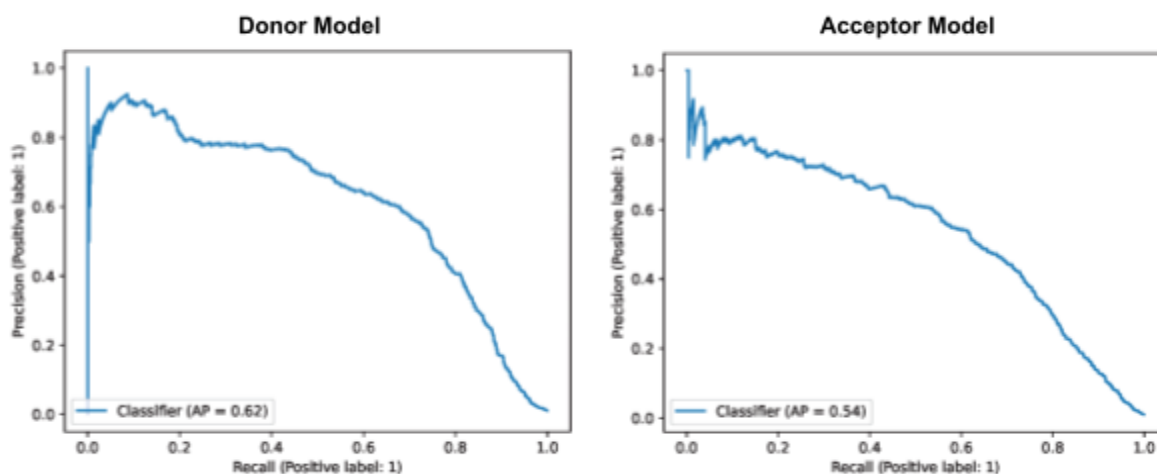


Figure 16. Precision-recall curves for bidirectional LSTM models for donor sites (left) and acceptor sites (right) with the area under the curve (AP, 0.62 and 0.54 respectively).

CNN/LSTM: From a hyperparameter search evaluating the best set based on F1-macro scores, we found the best performance for the hyperparameters in **Table 13** for a combination CNN and LSTM model. The donor and acceptor models had a different optimal number of filters.

Table 13. Best hyperparameter values for CNN/Bidirectional LSTM models.

Parameter	Donor Model Parameter Value	Acceptor Model Parameter Value
Initial learning rate	0.01	0.01
Hidden Size	100	100
Number of Filters	128	64
Kernel Size	9	9

Dropout Probability	0.25	0.25
FC Size	128	128

The CNN/LSTM model for donor sites had a test accuracy of 99.1% and the one for acceptor sites had a test accuracy of 98.9%, which is around the same as the LSTM models. **Figure 17** shows the ROC curves for both models with AUC scores of 0.98 for both models, an improvement compared to the LSTM models. The confusion matrices for these models on the test set are in **Table 14**. The F1-macro score for the donor model is 0.81, the same as the bidirectional LSTM donor model. The F1-macro score for the acceptor model is 0.77, a slight decrease compared to both LSTM models. The precision-recall curves are shown in **Figure 18**.

Table 14. Confusion matrices CNN/LSTM.

Donors	Predicted False	Predicted True	Acceptors	Predicted False	Predicted True
False	67,537	448	False	81,564	780
True	181	518	True	148	572

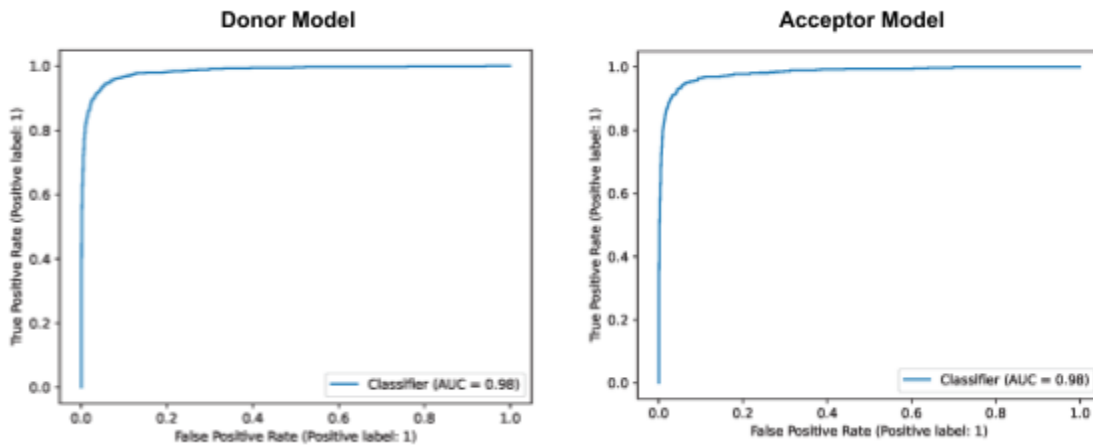


Figure 17. ROC curve for CNN/LSTM models for donor sites (left) and acceptor sites (right) with AUC (both 0.98).

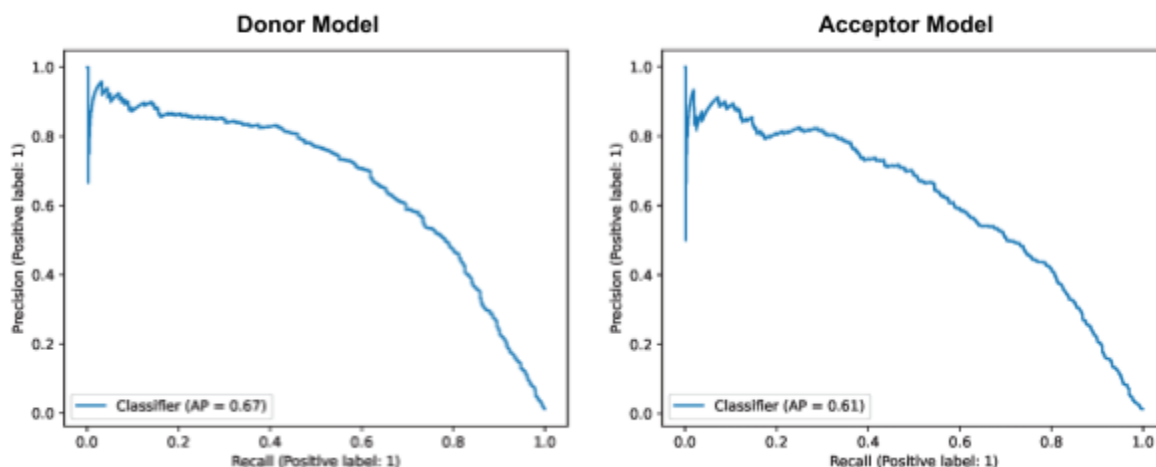


Figure 18. Precision-recall curves for CNN/LSTM models for donor sites (left) and acceptor sites (right) with the area under the curve (AP, 0.67 and 0.61 respectively)

We next visualized the weights for the first convolutional layer of the CNN part of the model, again to identify any enriched motifs driving donor or acceptor splice site activity. Note that as the first convolutional layer of this model contains 128 (donor) and 64 (acceptor) filters, we display only the sequence outputs of the first few filters (**Figures 19 and 20**). The full sequences can be viewed in **Supplementary Table 2**.

Donors

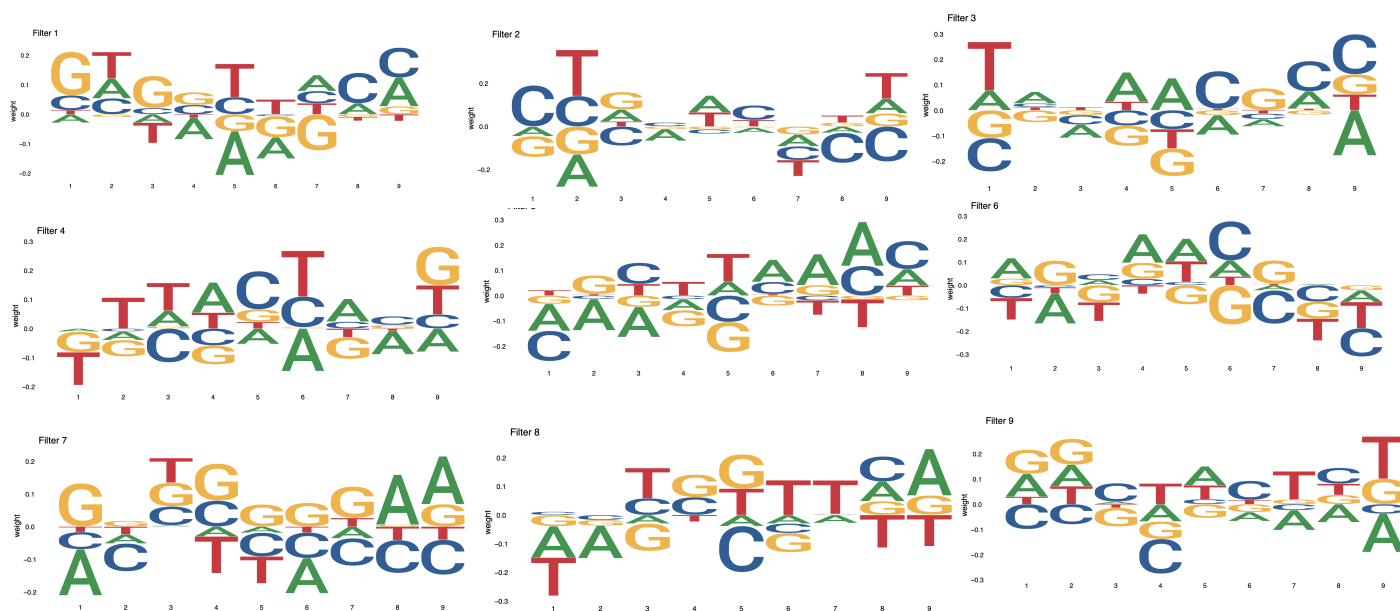


Figure 19. Visualization of various CNN filters for donor site CNN/LSTM model.

Acceptors

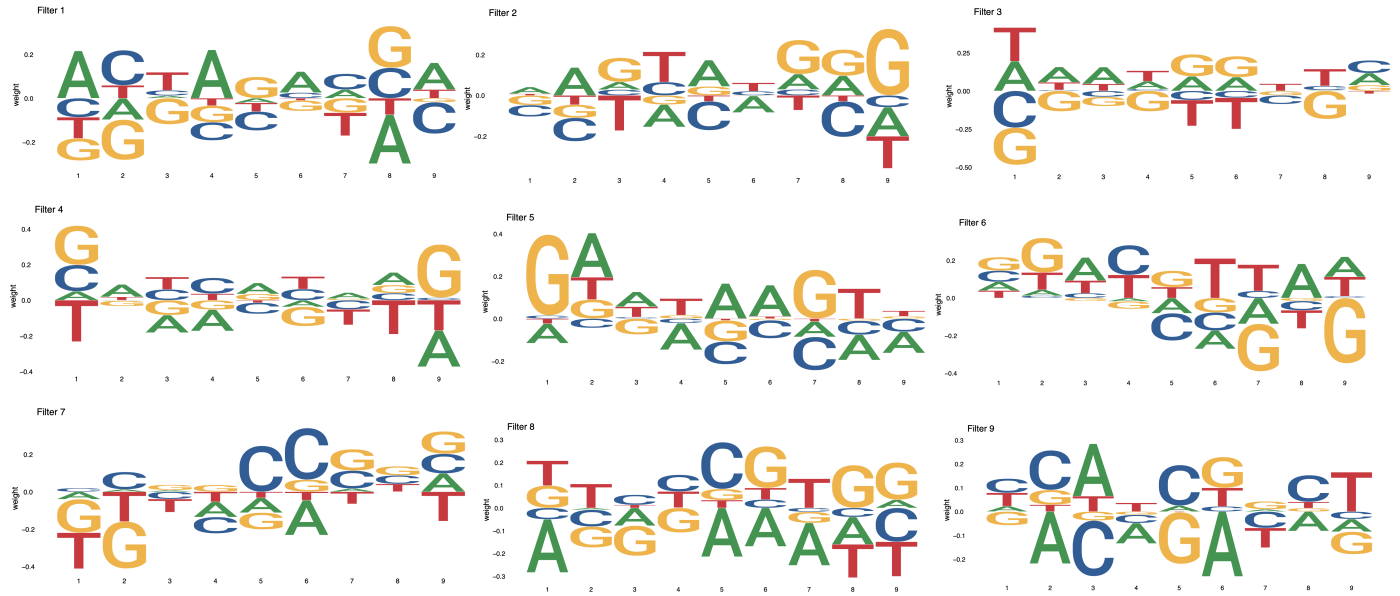


Figure 20. Visualization of various CNN filters for acceptor site CNN/LSTM model.

We next performed enrichment of the motifs present in these sequences prioritized by the CNN convolutional layer using the MEME suite of tools. The results are shown in **Figures 21** and **22**.

Donor

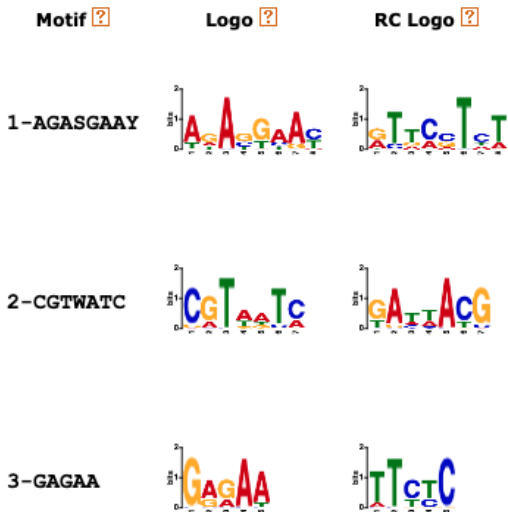


Figure 21. Motif enrichment of CNN filters for donor site CNN/LSTM model.

Acceptors

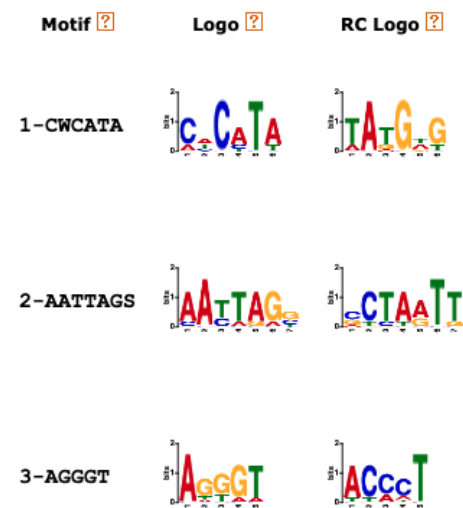
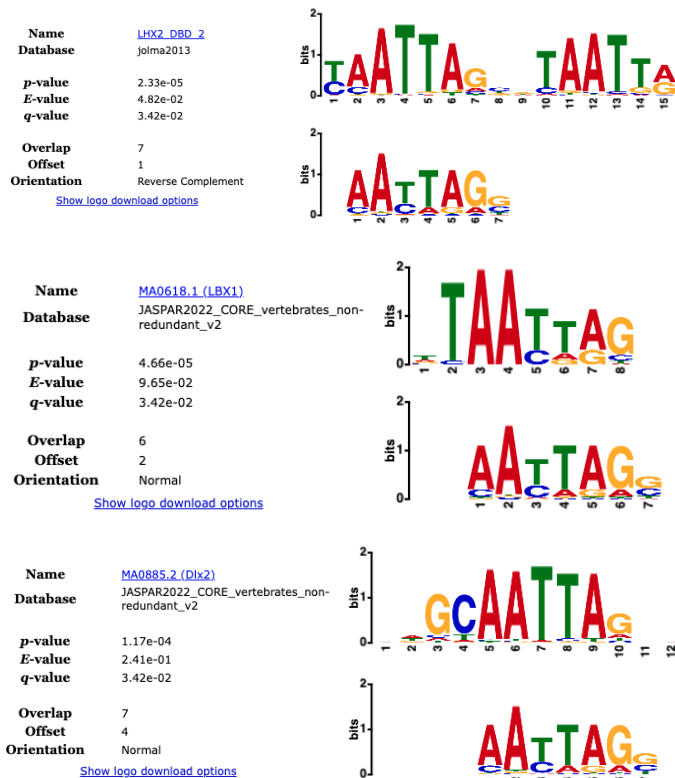


Figure 22. Motif enrichment of CNN filters for acceptor site CNN/LSTM model.

Notably, the second motif discovered in the acceptor sequences showed sequence similarity to a variety of known motifs, including the LHX family of transcription factors (LHX2, LHX4, LHX6, LHX8), LBX1, DLX2, En2, and HOXB7 (Figure 23).



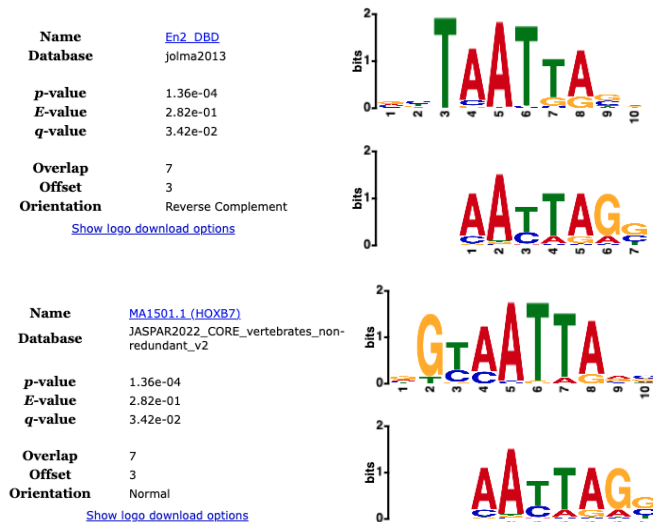


Figure 23. Known motifs that overlap the second enriched acceptor site motif.

LHX family transcription factors are known to play a developmental role in patterning of the nervous system and brain. LHX6 especially is associated with splice variants associated with brain development. LBX1 is similarly a transcription factor binding motif. Splice site variants in LBX1 have been associated with skeletal muscle and spinal cord defects such as scoliosis¹⁴.

The third enriched motif discovered in the acceptor sites showed sequence similarity to the known ZNF524 binding site (**Figure 24**). This binding site is also highly conserved across mammalian species with DNA binding properties, and multiple splice variants have been reported.

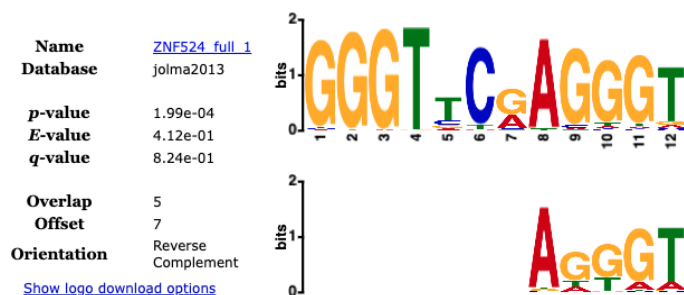


Figure 24. Known ZNF524 binding site that overlaps the third enriched acceptor site motif.

We then added max pooling layers to this model, detailed above. The hyperparameter search yielded optimal hyperparameter values shown in **Table 15** for the CNN/LSTM model with max pooling. This donor model has the same optimal hyperparameters as the donor model without max pooling above. The acceptor model has slightly different optimal hyperparameters. The donor and acceptor models have a different optimal kernel size.

Table 15. Best hyperparameter values for CNN/Bidirectional LSTM models with max pooling.

Parameter	Donor Model Parameter Value	Acceptor Model Parameter Value
-----------	-----------------------------	--------------------------------

Initial learning rate	0.01	0.01
Hidden Size	100	100
Number of Filters	128	128
Kernel Size	9	7
Dropout Probability	0.25	0.25
Pooling Kernel Size	2	2
Stride	2	2
FC Size	128	128

The CNN/LSTM models with max pooling for both donor and acceptor sites had test accuracies of 99.1%. **Figure 25** shows the ROC curves for both models with AUC scores of 0.99 for both models, an improvement compared to the LSTM models and the CNN/LSTM model without max pooling. The confusion matrices for these models on the test set are in **Table 16**. The F1-macro score for the donor model is 0.81, the same as the bidirectional LSTM donor mode and the CNN/LSTM model without max pooling. The F1-macro score for the acceptor model is 0.79, which is the highest score of all of the acceptor models. The precision-recall curves are shown in **Figure 26**.

Table 16. Confusion matrices CNN/LSTM max pool.

Donors	Predicted False	Predicted True
False	67,532	453
True	169	530

Acceptors	Predicted False	Predicted True
False	81,827	517
True	210	510

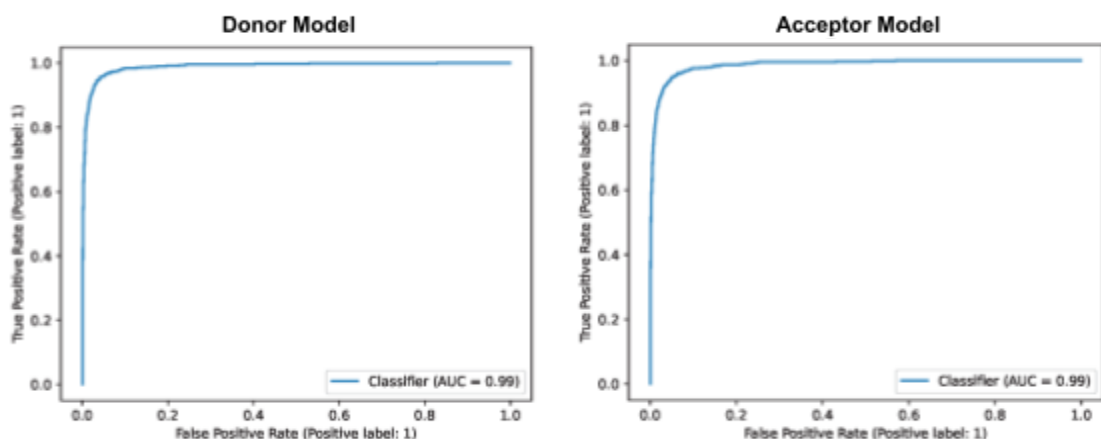


Figure 25. ROC curve for CNN/LSTM models with max pooling for donor sites (left) and acceptor sites (right) with AUC (both 0.99)

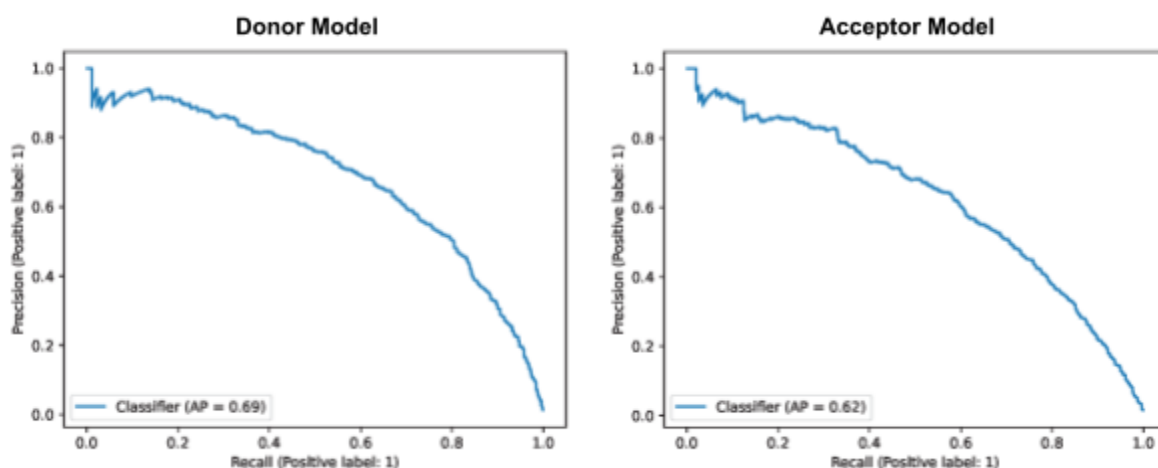


Figure 26. Precision-recall curves for CNN/LSTM models with max pooling for donor sites (left) and acceptor sites (right) with the area under the curve (AP, 0.69 and 0.62 respectively)

Again, we visualized the weights of the first convolutional layer to identify any enriched motifs driving donor or acceptor splice site activity. Note that as the first convolutional layer of this model contains 128 filters, we display only the sequence outputs of the first few filters (**Figures 27 and 28**). The full sequences can be viewed in **Supplementary Table 2**.

Donor

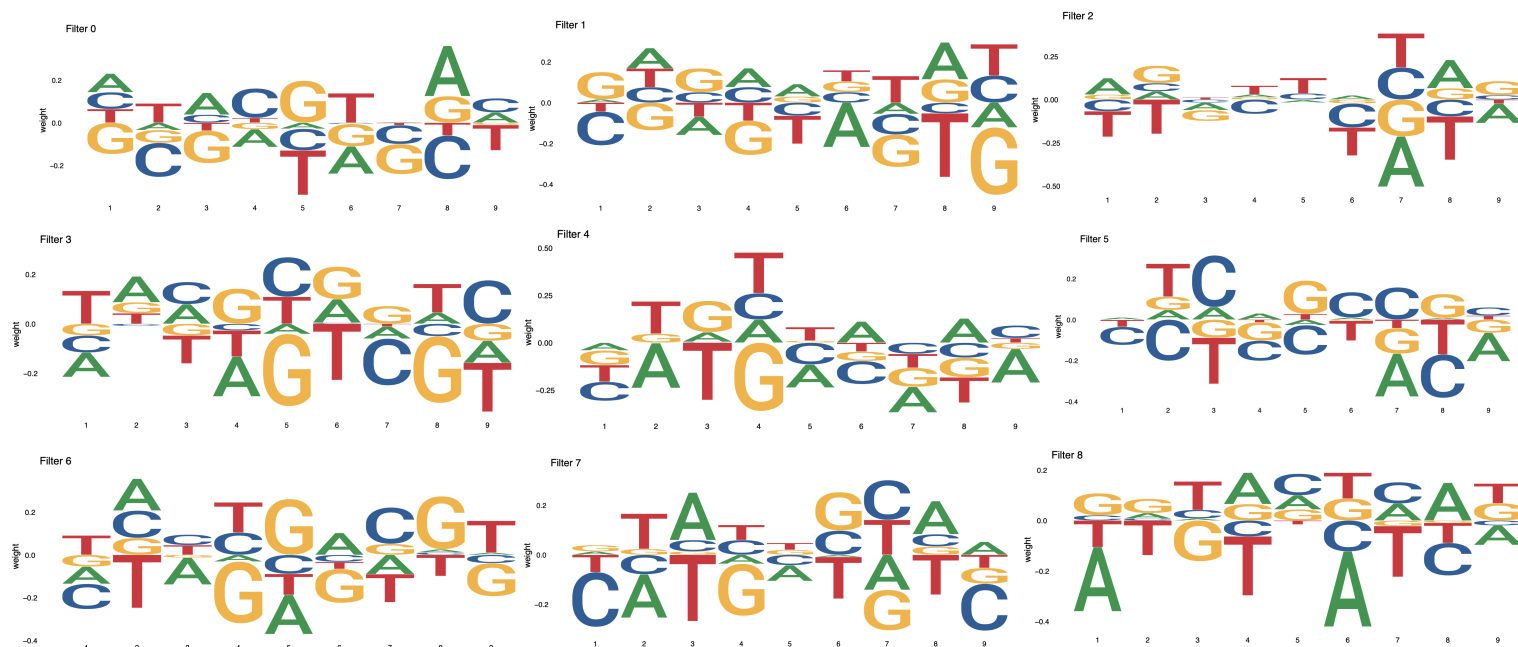


Figure 27. Visualization of various CNN filters for donor site CNN/LSTM model with max pooling.

Acceptors

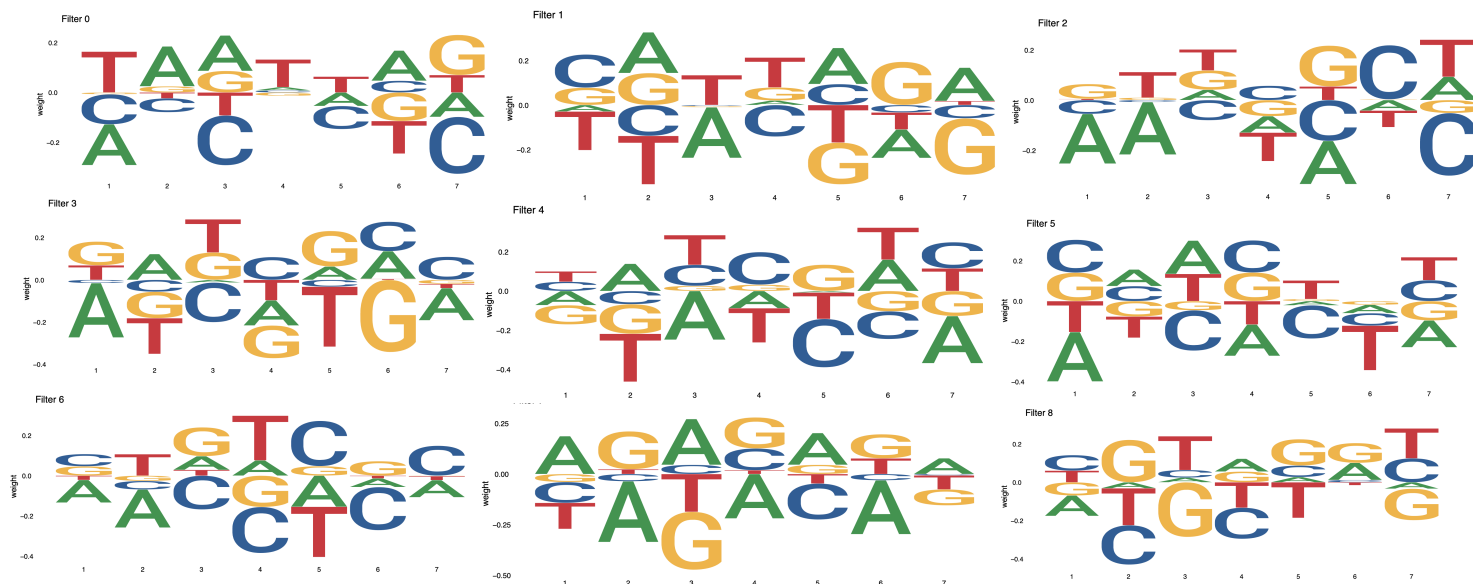


Figure 28. Visualization of various CNN filters for acceptor site CNN/LSTM model with max pooling.

Finally, we used the MEME suite to assess enrichment of the motifs present in these sequences extracted from the CNN convolutional layer (**Figures 29 and 30**).

Donors

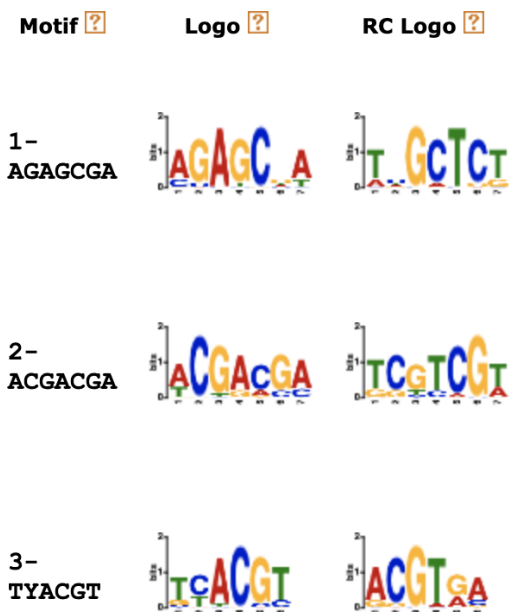


Figure 29. Motif enrichment of CNN filters for donor site CNN/LSTM mode with max pooling.

Acceptors

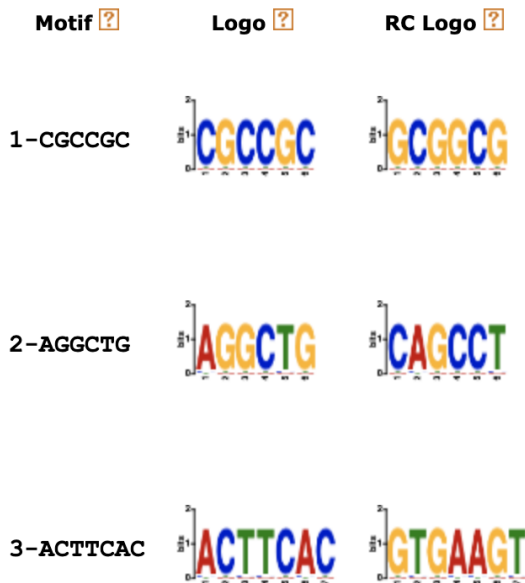
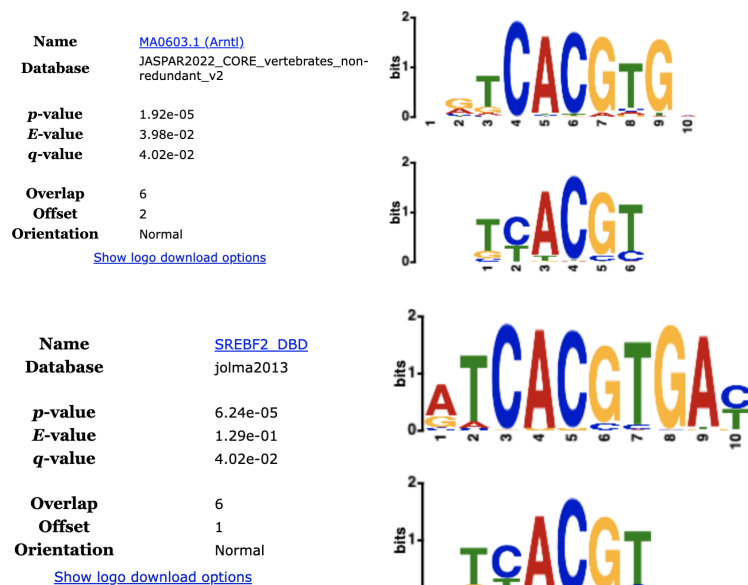


Figure 30. Motif enrichment of CNN filters for acceptor site CNN/LSTM model with max pooling.

The 3rd enriched motif in the donor sequences shows sequence similarity to multiple known motifs (**Figure 31**), including ARNTL, TFEB, MLX, GMEB2, USF2, TFEB, ERF::SREBF2, and the BHLH family, including BHLHE40 and BHLHB2. ARNTL (BMAL1) is a basic helix-loop-helix protein that forms a heterodimer with CLOCK, acting as a transcriptional activator for the circadian clock. The basic helix-loop-helix (bHLH) proteins form a large superfamily of transcriptional regulators that function in critical processes in eukaryotic development¹⁵.



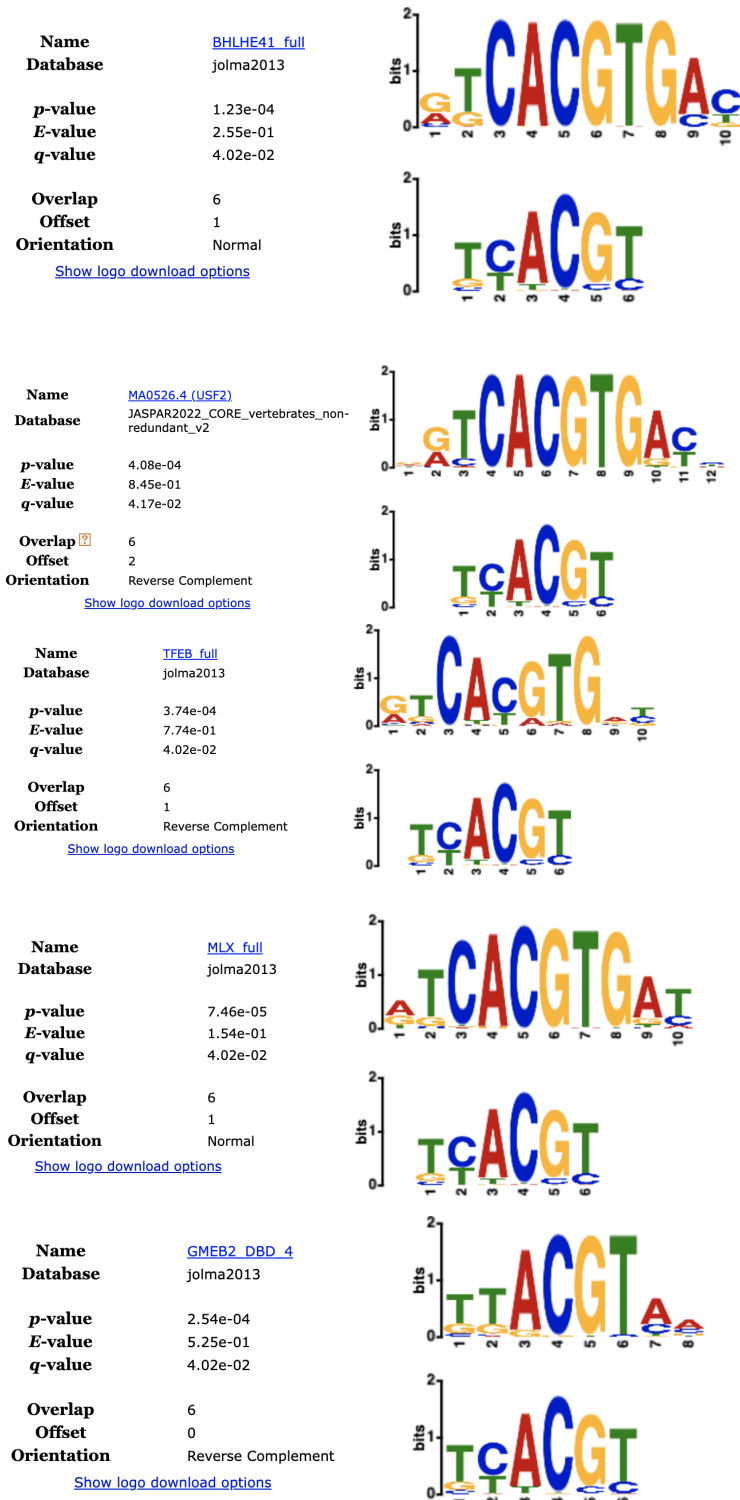


Figure 31. Known motifs that overlap the third enriched donor site motif.

Enrichments for the acceptor sequences included structural similarities to ZNF610 and ZNF460 (Figure 32).

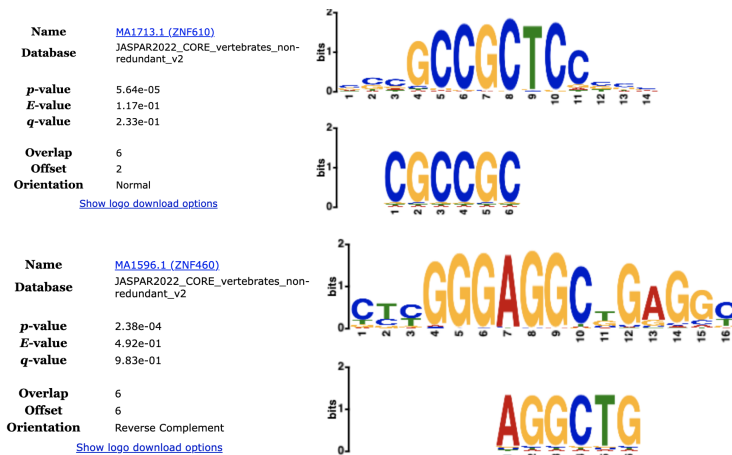


Figure 32. Known motifs that overlap enriched acceptor site motifs.

Discussion:

Of the models tested, the best performing models based on the F1-score were the LSTM based models. For donor splice sites, the bidirectional LSTM model, CNN/LSTM model and the CNN/LSTM model with max pooling all achieved an F1-macro score of 0.81. The best model of the three depends on what you want to optimize for. The bidirectional LSTM had the highest accuracy (99.2% vs 99.1% for the other two). However, the CNN/LSTM model with max pooling had the highest AUC (0.99 vs 0.97 for the bidirectional LSTM model and 0.98 for the CNN/LSTM model) and the highest recall (0.76 vs 0.70 for the bidirectional LSTM model and 0.74 for the CNN/LSTM mode). For acceptor splice sites, the best model was the CNN/LSTM model with max pooling with an F1-score of 0.79. The ability of LSTMs to process sequences was likely important in classifying the DNA sequences as the LSTM models performed better than models without an LSTM layer.

One way to potentially improve the models we used would be to change the embedding layer in the LSTM models. In this case, we treat each base as a “word” in the embedding. It might perform better if we treat a k-mer (a sequence of k bases) as a “word” instead of only one base at a time. While the final accuracies are generally very high, even with our best model we still predict a significant number of false positives and false negatives. In order to further improve this, in the future we could test alternative methods for handling class imbalance. For example, we could try Synthetic Minority Oversampling Technique (SMOTE). SMOTE generates synthetic data for the minority class.

Elucidation of splice sites within nucleotide sequences is a biologically essential task to understand the mechanisms underlying regulation of gene expression, and how DNA sequence variants may lead to disease. Here, we offer various machine learning strategies for recognition of donor and acceptor splice sites, that can detect motifs which are most likely to play a role in determining the activity of a splice site. Machine learning strategies are promising to identify sequences that previously have not been associated with donor or acceptor splice site function, as they are unbiased to prior assumptions regarding splice patterning. However, this also represents a weakness of machine learning approaches. Here, inputs to our models were delivered in the form of 1D sequences of nucleotides. However, in reality, DNA sequences interact in 3D space, forming loops with regulatory patterns that are not represented by 1D

sequence representations. Further studies assessing splice site mechanics should aim to incorporate known information about 3D structures, such as integrating information from multiple functional modalities, such as Hi-C mapping and CHIPseq. Of course, this is a challenging task, as 3D chromatin mapping information is rare and context-specific, making it difficult to collect. The quantity of data required to train machine learning models may make such models difficult to develop.

Despite the weaknesses of a 1D model, we were able to devise multiple approaches that identified both known and novel motifs likely to play a role in splicing activity. We achieved this by utilizing convolutional neural networks (CNNs). While CNNs have largely been used for image recognition tasks on 2D images with 3 (RGB) channels, we represented nucleotide sequences as 1D vectors with 4 (A,C,T,G) channels, allowing for ease of interpretation of the first convolutional layer. When we plotted weights from the first convolutional layer, we were able to determine sequences identified by the CNN as relevant to the classification task, as either positively or negatively associated with the positive class label (here, donor or acceptor splice sites). Each filter identified a slightly different nucleotide sequence that was positively associated with splice sites. To determine motifs common to splice sites, we used already developed tools for motif enrichment and comparison to known motifs, such as the MEME suite. We were able to identify enriched motifs that are binding sites known to play a role in splice regulation (eg: LBX1, PAX, and CENPB), as well as novel motifs identified by the CNN that may represent novel motifs that regulate splicing. These enrichments show the ability for machine learning models to identify motifs relevant to splicing in 1D input sequence data.

Overall, we were able to achieve high accuracy with our donor and acceptor splice site predictor models and reasonable F1-macro scores by testing a variety of model architectures and hyperparameters. While there is room for improvement at reducing false positives and false negatives, these models are generally successful at predicting splice sites.

Supplemental Table 1: Sequences of all motifs prioritized by first CNN convolutional layer in best CNN-only model.

Filter	CNN	
	Donor	Acceptor
1	ACTTTATTGTCGAGT	AATGTTGAAACGGAC
2	GGGCGGGGGGGGGGGG	AGGTAGGAGAGGGGA
3	CTGGCCGGGTTCAC	GCTAATGGCCTAACT
4	GTTTACAAGCGTTAA	AGCTTCACGAGCAAT
5	AAGTTTAAATCTCAT	AGGAGACGCTAGACT
6	CTCAGTAACGTGTCA	AATTTCAAGCAGAGG
7	CCAACATCGGCTGCA	TGTCATGATGAGTTT
8	CACGAATCGTGAAAT	CACTTGTTTTCTCCC
9	GGTCGTTGAGCGAGC	TAATAATTATCTCAC
10	GTCGACGACGACGAC	TAAGTAGTTTAGGTC
11	GTGAAGCAGCAAAGT	ATGCTGCTGATCATC
12	CTGCGCGGTACGGA	AGCTGAGTCTCAACT
13	GTCTATTGATTGGCG	ATCTTGATAGGTAGGT
14	TGTTAGTTTAGGTTT	CATGCAAGAGATGAA
15	CAAGGTAAGAGTATC	ACTCAACGTAGGTTT
16	CCTCTAGCCGACTCC	GGGTCCCCGCTCGTG

17	AGGTTAGTCATCAGC	ATAGCAAGATAGGTT
18	TCGTGCTAAGAGTGG	TAAGTTATCCTAATT
19	TAAATAGTTAGTTAG	GTTTTTCATTGCATG
20	GATAACAAGTCAAGC	TCTTCTATGCGACGC
21	CGAGAGAGTACGAGC	TTCACATAGTAAGCC
22	GTGAGTTGAGTATTG	ACAGGGCAGGTGTCT
23	ACTGTGGATCAAAGT	TAGCTCTGTCCCCGG
24	ATAGTAACTGCCCTC	ACGGATCGAGGCGAC
25	TGAGGTACTTCGCGA	AAGTAGGCAAGCGCA
26	GAAGATTAGCTTGGT	CGACGACTTCGTCTGA
27	TAAGTCTCTTCATAG	GGCAGGAATAAGGAA
28	GATCGAGTAGAACGA	CTTACACTGATCCTA
29	CTAGACACGTAAGTC	ACAGGCGCTTTGTCTG
30	GGCAGGTACGTTATC	ACCAACTGCATCGCC
31	AGGCATGACCAAGTA	GTACAGATGAACGAT
32	ATACAGAGGTGTGCC	TGCAGATGCTGCACG

Supplemental Table 2: Sequences of all motifs prioritized by first CNN convolutional layer in best CNN/LSTM model

Filter	CNN/LSTM without max pool		CNN/LSTM with max pool	
	Donor	Acceptor	Donor	Acceptor
1	GTGGTTACC	ACTAGACGA	ATACGTAAC	TAATTAG
2	CTGCACGTT	AAGTATGGG	GAGAATTAT	CATTAGA
3	TATAACGCC	TAATGGTTC	AGTTTATAG	GTTTCGCT
4	CTTACTACG	GATCATAAG	TACGCGGTC	GATCGCC
5	TGCTTAAAC	GAATAAGTT	ATGTTACAC	TATCGTC
6	AGCAACGAG	GGACGTAA	ATCAGCCGC	CAACTGT
7	GGTGGGGAA	CCGGCCGGG	TACTGACGT	CTGTCGC
8	CTTGGTTCA	TTCCCGTGG	GTATTGCAA	AGAGAGA
9	GGCTACTCT	CCATCGGCT	GGTACTCAT	CGTAGGT
10	TTGGCTCCC	CGTGGTCAT	AGTCTGTAC	TAGATCA
11	TCCACTGTG	TTGATCTCC	TGCGTACGT	GCAGTGA
12	TGACTGACG	CTGCGATTT	GAGCGTCGA	GCGCCGT
13	CCTGGGAAC	TCTACCACA	AGAGCATGA	ACTACAC
14	AGTCGCGGT	CGTCACTTT	GCTACGTTA	GAAGTAGT
15	CGGAGCATC	GAGCTAGAT	AACGTATAA	TTCGGGT
16	GGAAGGGCC	ACTAAAAC	TCGACGAGA	CGTATCA
17	AGATCAGCT	CGTAAAAAC	TATGCACAC	CGCCGCA
18	AAGAAATTA	TTACATTGT	ATATCAGTA	AGTTCTGA
19	GCAGTTCGG	CAGGGAGTA	CCTTAAACG	GCGGTAG
20	CGTTTTCCA	GTACGTAGA	GCGTAGGTG	GGAGCTT
21	GACGTAATC	CGACGTAAA	GACCGCGAG	GAGCGCG
22	CCCTGTGAT	GCGCTAGTT	CGCTTTCCA	ACAACGC
23	TGGGAGCTA	TGAGAAGTT	ACTCCACCA	CGCAGCT
24	GTCTCGCGA	TTTGTCGTT	TGACAGGAT	TGACGTC
25	GACGAGCGC	TAGAGCGGG	GGTGATATG	AGTATGA
26	ACGAATTAA	CACTGTCCA	TCGTCTGTG	CGTGAGT
27	TCTGGAGTC	TCCTGTAGA	TAGTGGTGC	AGTCGAG
28	TTTAGTCTGA	ACGGACAAG	TCGACCTCG	TATGTCT
29	GAAGAGAAG	CCAGCGCTG	CAGTCTCGT	GCACCAG
30	TATTCGTTT	CACGTGCGG	AACGTGCGC	TAGTTAT
31	TTGATCTCT	TACGGTTTT	CGGATGAGC	TACAAGA
32	CTCTAACTC	CGGACCATA	ACGACGACG	AGTGTCA
33	TGATGAATT	TGAATGAGG	TCTCTTAGT	ACGCTTA
34	TAATCAGTT	GACGGTTGG	ACGGCGCTG	GACATAG
35	TTTGGAAGT	TGCAACACT	ACGACGACT	ATCCGAC

36	GGGGTTTGT	CTCGATCCA	GACTCGTCT	GAATGAG
37	CAAGTCGAC	GCGGCTGGG	CGTCGCTCG	AGATCGA
38	AAAGGAATC	TAGTCGGGG	ATAACTTAA	GACCAGC
39	CACGTCTGA	ACAGGATAG	GTATACTAG	CAACGTC
40	TTACGAATG	GTAAGGTGA	TCGAGCTAA	TGCGACA
41	TCGCGTACG	CGAAAAAAC	ATGGCGTAA	GAGCATG
42	TGTCATTTT	GGCCAGGGT	GTGAGTCTT	GCAGATA
43	AAAGGCAAA	GGAATCGAC	CGACGTGAA	ATATGAG
44	CACGCTACC	CACCTAATT	TGGTTAGTC	CATACGC
45	CTAAGCTTT	TGTATGTGA	TCTACTCAT	GTAACCT
46	GTGACTCTC	TTACGCGTA	CGACACTGC	CGAGGAG
47	AGTTAACGA	TAGTTTCTG	CGCAACATC	CTGCTCA
48	CATGTGTCC	CAAGTCTAC	CACACATAC	CGTATGC
49	CGTTCGTCT	ACTGACGGG	GAGACGAGT	GTAATAT
50	TGTTAAAGA	ATGTGCTTG	CCAGATCCA	TATCCAA
51	CCTGGCATA	TGCTCGAAC	GTGGCGGCG	GGACGAG
52	GCAAGACGA	AACATTGGG	TAGAGCTAG	GTGAAGT
53	GATACTCCG	ACGAAAGTG	CCAAATGAG	GATGGAG
54	GATTATGAT	GACCACGAA	GGAATAGTG	GGCGCGG
55	AAGGAGAAG	AATAGACAA	TCTACGTGA	AGTGAGG
56	ATTCGAGGC	GCTTCTGGG	GTAGCTCGT	CGACATC
57	TACTGACGC	TCTGCGGGT	GCAGGCAGT	GATTCCG
58	ATGACTCGA	ACTCCAGG	TGAGGTACT	CTCTACG
59	CCCCGTATA	CACTTAATT	TACGACCTC	AATCCGC
60	ACGACCCTA	GGGTCGCAA	CCCCGGTAG	TAGTAGG
61	GTAAGGTGA	CGCCGGATA	CAGCCAGTA	TAGCTCA
62	GTACCTATA	ATTACCACC	CTTACCTAG	TGCAGAT
63	TGGCGACGG	GATCAAGTA	TATGTCATC	CCTGCTG
64	TATACCGAT		CGCGACAGG	ACGAGGA
65	AAAAGGTTT		CTGTACAGA	GCGTGAG
66	GGAGCTAGT		TAGTATTGA	CCGCGAC
67	TACGTCAGG		TCTAGCTAC	GGGAAGT
68	TTTTGTGAA		TTAGTGTAG	GCTTCTG
69	TGTGCCCCA		TACTTAGGT	TTATGAT
70	GAAGAGCGA		TTGTTCTGT	GCAGGCG
71	TGAGGAGTA		TATACAGCA	AGCAGAG
72	CGACCAGTT		CGACCACTC	TCTTAGT
73	GGACACGTG		ACCGAACGT	TGTAATC
74	TCGTGTTGG		GAGTAGCAC	GCTCTCA
75	CGTCGTCGT		GCCTCGCAG	AAGGCTG
76	GATGTGCCG		GCGTCGTAA	AGTAGGC
77	TCGCGGCTA		TGCGCGTCG	CAGTAGA
78	CGACTATCT		GCATCTTGC	TCATCAC
79	GGCAACGGC		ATGCTAGTT	CGACGAG
80	TAACGCGGC		CTATCGTCT	GCGCGAC
81	CCCCTCGA		TCGTGCACT	CCTATCC
82	ATTAGACAT		CCAGCCGAA	GTCGTGC
83	TCGTGGCCC		TAGGTACGT	CACTATA
84	TGGACTCAA		ATCAAGGCA	CTCATTC
85	GGGAGGGAT		GCGCGATAA	GCTAGAC
86	AGGATCGTC		TCGCTCTAA	ACGACTT
87	ATGCGCCGC		TGACGTCAA	ACAGTGT
88	CGACCACGA		TTAGTGACA	TAACGCT
89	AAAGCCTCG		CATGAACGA	CGTCGTG
90	GATGTTACG		CTCGACGTC	GAGGCGG
91	GACGAATCT		GTCGCGACA	AGGTAAG
92	AGACCGCGA		AGCATGTGT	TTGCAAG
93	TAAAGAAGC		TCGTCACGC	ACACCGT

94	ACGCATAGG		AATAGTACT	CGTACAA
95	CAAGTAGCT		CAATGTCAT	ACTCATG
96	CCCAGGAGC		CGATTAGAC	CCTACGA
97	ACGGACCCC		AATCCATCG	CGTCATC
98	AAGTCTAGG		CGATGCGGT	AAGTCAG
99	AACCGAATT		TTATGCTAA	AACTAAT
100	TCATTGTAG		ATACAATAG	ACGAGAA
101	TATTTACCA		ACGTAGCAG	TCGTTCT
102	TTTACGGAT		CGCAGATAA	ACTACGC
103	ATCTTAGCG		CGACGACGA	TGAGTCG
104	CACAACCCT		GCGAATATC	ATGGTCA
105	GGTCCCGAC		ACGGTGCTA	TCACATA
106	TTCTAATCG		CTATGGCGT	TGCGTAC
107	CAGGGTCCC		CATCGGAAG	TCATACT
108	GCGAAAAC		CGTCACCTT	AGATAGG
109	AGATTATCC		GTTACGTAG	AGTACGA
110	GGGACGCGC		ATCGGCCGC	ACCAACG
111	GAAACTTAG		GTCAGTTAG	TATACGG
112	GGTCGTCCC		CATATGAGT	GATGATA
113	GCACTCACA		GTCAGCAAG	CACATGC
114	CACTTATAC		TTATCTCCG	TATATCG
115	GAGCGGCGA		TAACTAGGA	TGCACGA
116	ATCAGCCAC		GTCGAATGC	CATAATA
117	GGCCGTACC		GATCGCTGT	AGCTCGG
118	AGGACGTGT		GTCTATACT	GATGAGG
119	AAACAAGAT		AAGTATTAC	GTACGTG
120	CTACCTCGA		GTACCACGT	AGGACGA
121	CTTGGGTAG		GCGGTACTA	TATATAT
122	AAGGAGGGT		GACGGCTTC	AGGGTGC
123	AAGTTCCCC		CTTGCGTCA	GTATAGT
124	GGAATAACT		ATGCACCAC	GACTAGA
125	CCCCGACGA		GATTCAAGT	CTGTTAC
126	TGCCTCGAA		GTAATCGTA	TTAGATA
127	CCGCCCAT		CTCCGTTCA	ATAGTAG

References

1. Burset, M., Seledtsov, I. A., & Solovyev, V. V. (2000). Analysis of canonical and non-canonical splice sites in mammalian genomes. *Nucleic acids research*, 28(21), 4364-4375.
2. Zamore, P. D., & Green, M. R. (1989). Identification, purification, and biochemical characterization of U2 small nuclear ribonucleoprotein auxiliary factor. *Proceedings of the National Academy of Sciences*, 86(23), 9243-9247.
3. Smith, C. W., Porro, E. B., Patton, J. G., & Nadal-Ginard, B. (1989). Scanning from an independently specified branch point defines the 3' splice site of mammalian introns. *Nature*, 342(6247), 243-247.
4. Stern, R. C., Doershuk, C. F., & Drumm, M. (1995). 3849+ 10 kb C→ T mutation and disease severity in cystic fibrosis. *The Lancet*, 346(8970), 274-276.

5. Busslinger, M., Moschonas, N., & Flavell, R. A. (1981). β^+ thalassemia: aberrant splicing results from a single point mutation in an intron. *Cell*, 27(2), 289-298.
6. Cartegni, L., Hastings, M. L., Calarco, J. A., De Stanchina, E., & Krainer, A. R. (2006). Determinants of exon 7 splicing in the spinal muscular atrophy genes, SMN1 and SMN2. *The American Journal of Human Genetics*, 78(1), 63-77.
7. Garcia-Carmona, Y., Ting, A. T., Radigan, L., Athuluri Divakar, S. K., Chavez, J., Meffre, E., ... & Cunningham-Rundles, C. (2018). TACI isoforms regulate ligand binding and receptor function. *Frontiers in immunology*, 9, 2125.
8. Pollastro, P., & Rampone, S. (2002). HS3D, a dataset of Homo Sapiens splice regions, and its extraction procedure from a major public database. *International Journal of Modern Physics C*, 13(08), 1105-1117.
9. Zeng, H., Edwards, M. D., Liu, G., & Gifford, D. K. (2016). Convolutional neural network architectures for predicting DNA–protein binding. *Bioinformatics*, 32(12), i121-i127.
10. Grant, C. E., & Bailey, T. L. (2021). XSTREME: Comprehensive motif analysis of biological sequence datasets. *BioRxiv*, 2021-09.
11. Bogutz, A. B., Oh-McGinnis, R., Jacob, K. J., Ho-Lau, R., Gu, T., Gertsenstein, M., ... & Lefebvre, L. (2018). Transcription factor ASCL2 is required for development of the glycogen trophoblast cell lineage. *PLoS genetics*, 14(8), e1007587.
12. Thompson, B., Davidson, E. A., Liu, W., Nebert, D. W., Bruford, E. A., Zhao, H., ... & Vasiliou, V. (2021). Overview of PAX gene family: analysis of human tissue-specific variant expression and involvement in human disease. *Human genetics*, 140, 381-400.
13. Jiménez-Ávila, C. E., Villegas-Ruíz, V., Zapata-Tarres, M., Rubio-Portillo, A. E., López, E. I. P., Zenteno, J. C., & Juárez-Méndez, S. (2018). Centromere-associated protein E expresses a novel mRNA isoform in acute lymphoblastic leukemia. *International Journal of Molecular Epidemiology and Genetics*, 9(5), 43.
14. Takahashi, Y., Kou, I., Takahashi, A., Johnson, T. A., Kono, K., Kawakami, N., ... & Ikegawa, S. (2011). A genome-wide association study identifies common variants near LBX1 associated with adolescent idiopathic scoliosis. *Nature genetics*, 43(12), 1237-1240.
15. Jones, S. (2004). An overview of the basic helix-loop-helix proteins. *Genome biology*, 5, 1-6.